MASARYK
UNIVERSITY

FACULTY OF INFORMATICS

Musikk: A Modern, Social-First Music Streaming Platform

Bachelor's Thesis

KIRILL VOROZHTSOV

Brno, Fall 2025

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Musikk: A Modern, Social-First Music Streaming Platform

Bachelor's Thesis

KIRILL VOROZHTSOV

Advisor: Mgr. Luděk Bártek, Ph.D

Brno, Fall 2025


FACULTAS ARTIS INFORMATICÆ
UNIVERSITAS MASARYKIANA

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, the following AI tools were used:

- **ChatGPT**: Used for debugging and code improvements, as well as LaTeX formatting.
- **Grammarly**: Used for text correction.

Kirill Vorozhtsov

Advisor: Mgr. Luděk Bártek, Ph.D

Acknowledgements

I would like to thank Mgr. Luděk Bártek, Ph.D., for his time and the valuable advice he provided for this thesis.

Abstract

This bachelor's thesis aims to develop a web-based music streaming platform focused on social interactions between users.

The theoretical part presents a survey that analyzes people's music listening habits, showing that such a platform could attract a solid user base. It then compares existing music-related services with built-in social features and derives functional and non-functional requirements for the proposed system. Then, based on these requirements, suitable technologies and architectural patterns are selected.

The practical part covers the implementation details of the platform, including data model, audio processing, API design, and representation layer.

The result is a fully functional and deployed application.

Keywords

Audio Streaming, Python, JavaScript, MPEG-DASH, HLS, WebSockets

Contents

List of Tables

List of Figures

1 Introduction

Music streaming has been steadily growing in popularity each year for the past decade. [1] And this is to be expected; numerous studies have demonstrated how important music is for people. For instance, Rentfrow [2] suggests that music can influence mood, change self-perception, and help with social bonding. In addition, the rise of the clubbing scene, music festivals, and other social music-related activities clearly indicates the important social role of music. [3]

However, despite this social significance, it is surprising that features which facilitate social interactions are still not widely implemented in the existing music streaming platforms, as will be shown in **Chapter 3**.

The goal of this thesis is to design a music-centric platform that provides capabilities for collaboration and social interaction around music.

This work is divided into the following **six** chapters:

- **Chapter 2** Presents the outcomes of a survey illustrating how people consume music, how prevalent it is in social interactions, and why this type of platform can be relevant.
- **Chapter 3** Explores which social features existing streaming platforms and other music-related services already offer.
- **Chapter 4** Outlines the functional and non-functional criteria for the application.
- **Chapter 5** Describes the choices of technologies that are used by the application.
- **Chapter 6** Explains the development process and implementation details.
- **Chapter ??** Summarizes the results of the work.

2 Music Consumption Survey

In order to better support the choice of topic and demonstrate that a music streaming platform centered on social interactions could be a relevant service, a brief survey was conducted. It examined individual music listening and discovery habits, platform usage patterns, music-related interactions between people, and attitudes towards potential social features on the platform.

2.1 Methods

The service chosen for the questionnaire was Google Forms [4]; it provides a simple interface for survey creation, allows for easy sharing of the form, and supports extracting results as CSV files.

The questionnaire consists of 15 questions. Most of them are multiple-choice and closed-ended, with some allowing a custom answer. All custom answers are grouped under the "other" category in the provided figures. Answers in their original form can be found in the full survey.

The complete questionnaire is provided in Appendix A.1 and online.[1]

2.2 Results

This section presents the survey outcomes and analyzes their significance. Not all questions are discussed; for example, some questions dedicated to music discovery are omitted.

A total of 119 people participated, with the majority being in the 18 to 30 age group, with the exact distribution shown in Figure 2.1.

1. `https://docs.google.com/forms/d/1vhhAu_SfuHV4xy6JoDaRsM5uCElYn_csTmN8u4ZlpHc`

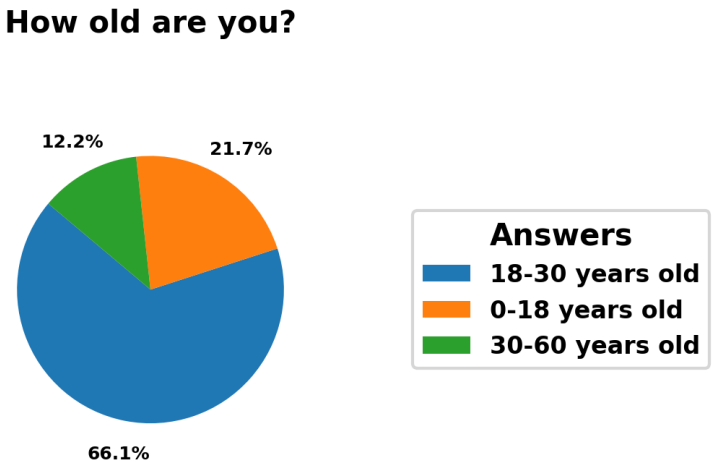

Figure 2.1: Age Distribution of Respondents

2.2.1 Engagement

Firstly, it was necessary to determine whether respondents listen to music frequently and whether they use streaming platforms for this purpose.

Over 50% of respondents reported listening to more than 500 songs per month, i.e., 16 songs a day on average (Figure 2.2), and nearly 80% stated that they listen to music daily (Figure 2.3).

The numbers clearly indicate that people have a significant interest in music, and the next logical step was to determine how the audio content is predominantly accessed.

2.2.2 Listening Methods

As shown in Figure 2.4, the proportion of respondents using streaming platforms accross all age groups is approaching 100%, with slightly higher numbers in the younger age groups. Although downloaded files and physical media are still used, especially among people aged 30-60, they are usually only a secondary option next to streaming solutions.

The question dedicated to the popularity of individual platforms has shown that Spotify is the most used one, which aligns with global

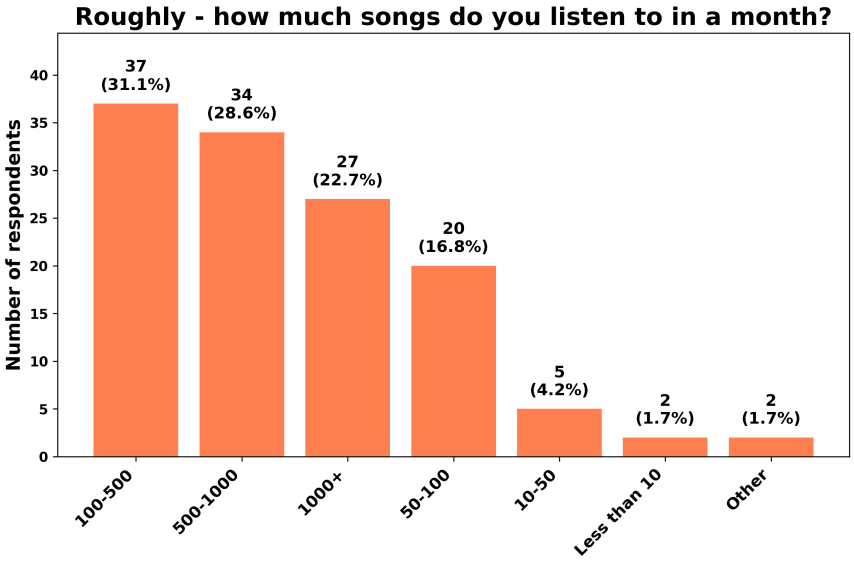

Figure 2.2: Monthly Song Count per Respondent

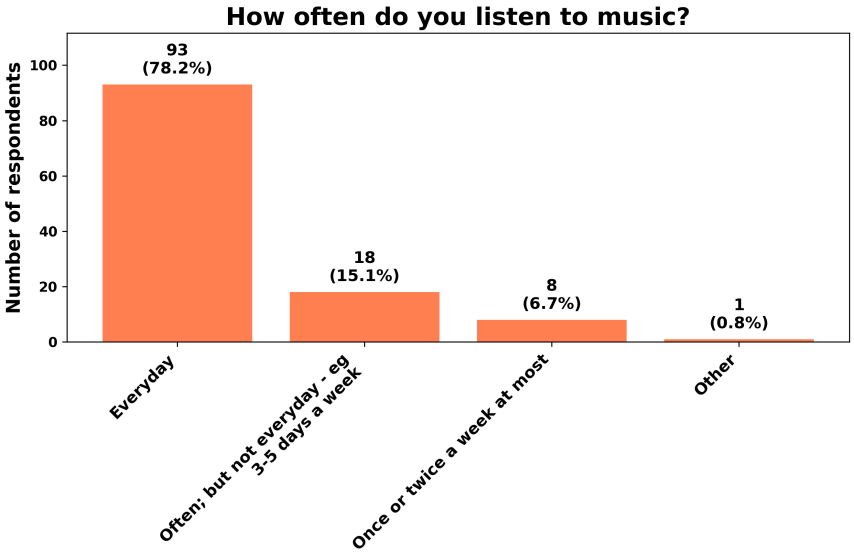

Figure 2.3: Listening Frequency of Respondents

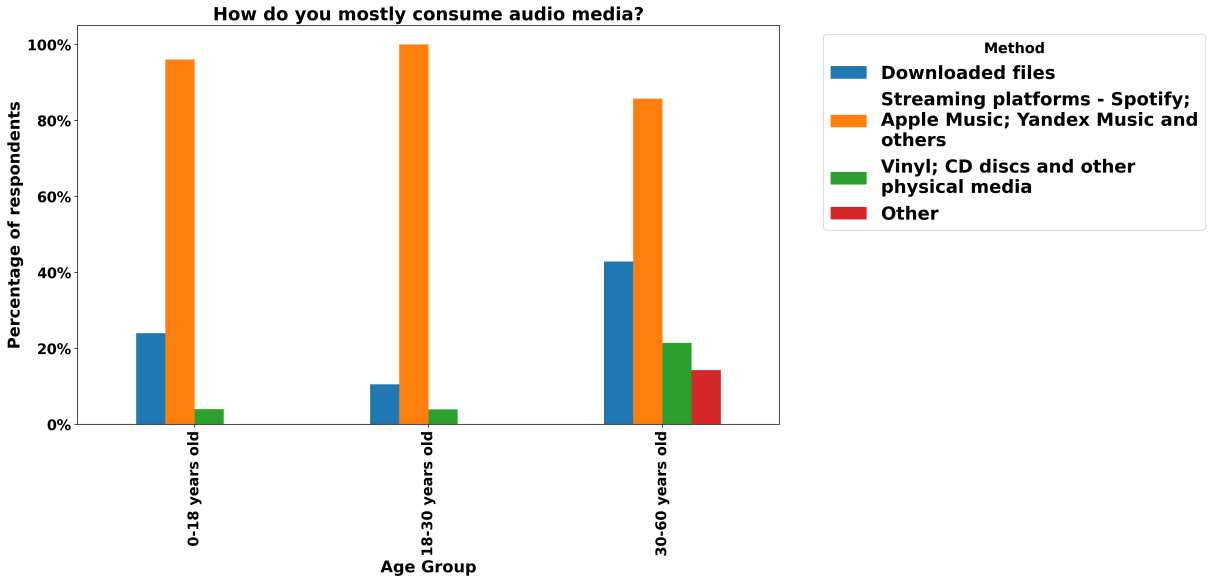

Figure 2.4: Audio Media Consumption by Age Group

statistics [5]. However, Yandex Music, coming in second place, deserves further explanation. Most of the respondents were from Russian-speaking countries, specifically Russia, and, with many Western companies leaving its market in 2022, including music streaming services, most users have moved to locally available products: Yandex Music, VK Music, and Zvuk. This is also the reason, why other popular region-specific platforms, such as Amazon Music (US), QQ Music (China), and JioSaavn (India), are absent from the survey.

The complete statistics are shown in Figure 2.5.

The survey showed that streaming services are indeed widely used. The next important step was to analyze the regularity of music-centered social interactions.

2.2.3 Social Interactions

The results of Question 10 indicate that nearly all respondents (99.2%) reported listening to music with others at least sometimes, primarily in offline settings such as gatherings or car rides (Figure 2.6). Online collaborative listening methods, although less common, were also mentioned by a quarter of respondents.

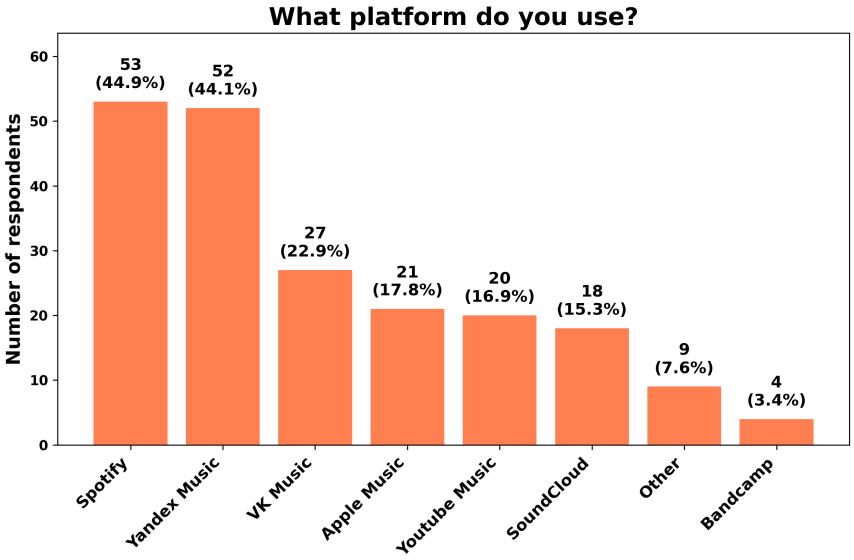

Figure 2.5: Streaming Platforms Popularity

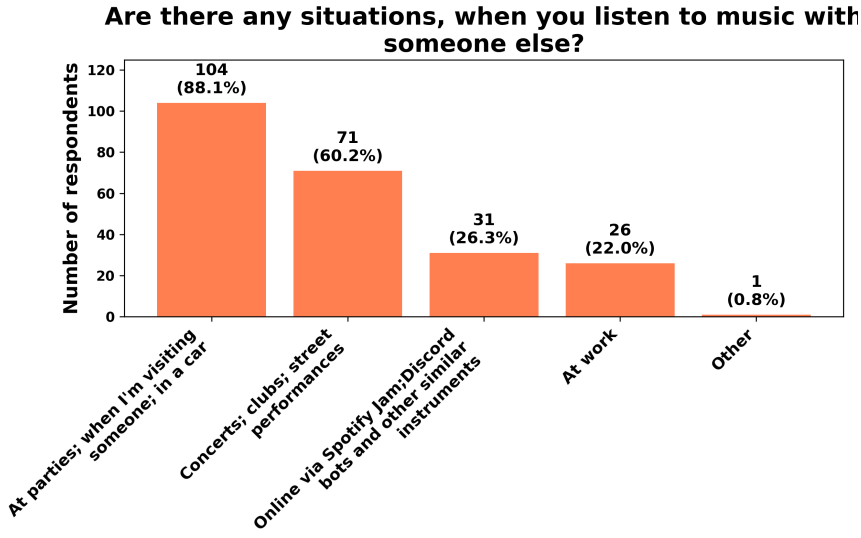

Figure 2.6: Social Interaction Methods

Question 11 (Figure 2.7) examines the frequency of these interactions: the results show that a significant portion of respondents engage in shared listening regularly. Around 60% reported doing so at least a few times per month, with a smaller group indicating almost daily interactions. This supports the previously mentioned idea in **Chapter 1** that music consumption is a common social activity.

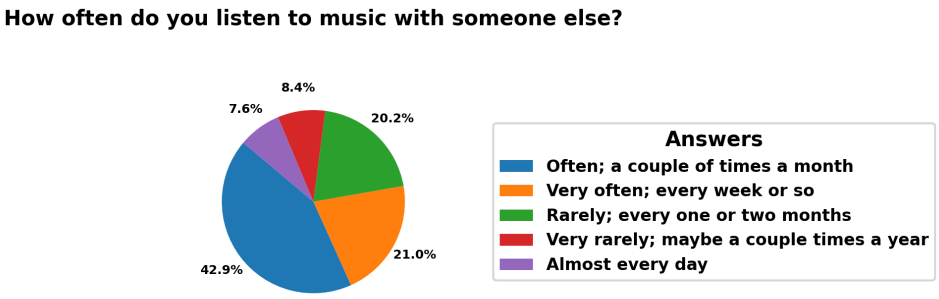

Figure 2.7: Social Interactions Frequency

The next question is phrased as follows: "How often do you/your friends share/discuss music?". While the previous question focused on real-time in-person collective listening, this one highlights more intentional and in-depth musical interactions, such as sharing songs, recommending music, or discussing it. Music, in that case, is not just the background but the primary purpose of engagement. The same trend as before can be observed in the responses: over 70% (Figure 2.8) indicated that they share music at least several times a month, with many doing so weekly or more often. This further emphasizes the active social role of music.

Question 13, "How does that happen?" (in terms of music sharing), asks about the specific ways people share music. Many respondents (Figure 2.9) stated that they typically send links from streaming platforms through external messaging apps. While this shows an apparent demand for sharing music online, it also reveals a gap among platforms, as these interactions remain fragmented. Hence, enabling such exchanges directly within the streaming app could provide a smoother and more uniform experience.

Responses to the question "Would you say that you know a lot of people with similar music taste?" were mixed, with a near-even split.

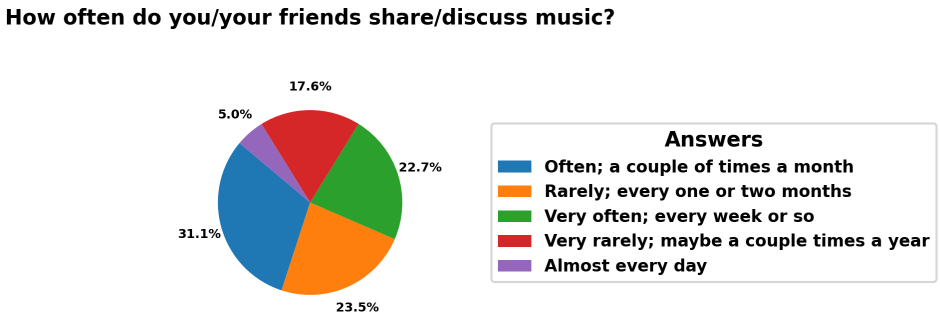

Figure 2.8: Music Sharing Frequency

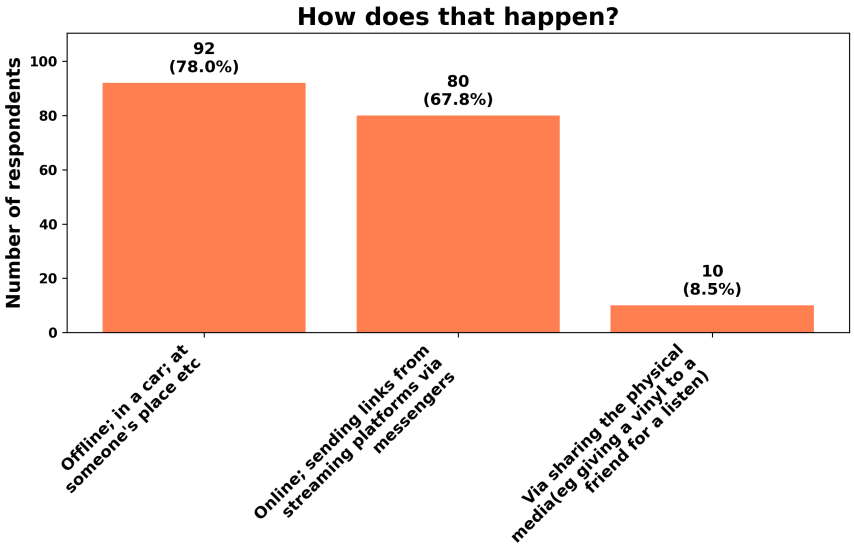

Figure 2.9: Music Sharing Methods

This suggests that while some users already have a social circle with similar music taste, many do not. In addition, when asked "Would you like to connect with others who share your musical taste, or influence those around you to explore and appreciate the music you enjoy?", the majority answered positively.

This data further support the idea that social discovery features could be useful within a streaming platform. Moreover, over 60% of participants stated in Question 15 that they would use additional social features if available on a streaming platform.

2.3 Summary

Overall, the results indicate that people listen to music frequently, with streaming services being their primary medium of accessing it. The data also suggest that music plays an important role in everyday life and often appears in social situations.

Yet, existing platforms only partly support interaction between users. This makes the idea of a music streaming service with built-in social features both relevant and likely to address real user needs.

3 Existing Platforms

In this chapter, existing platforms and services are examined to gain a better understanding of the instruments people use when interacting with music. Additionally, this chapter provides insight into the potential features that could be implemented in the resulting application.

Firstly, streaming services are discussed, as the main aim of the thesis is to create a system for music streaming. Then, other music-related services (e.g., forums) are presented, as they offer alternative possibilities for engaging with audio content that may be beneficial for the future platform. Lastly, platforms such as YouTube and TikTok are examined, because the survey has shown that they are a popular way to interact with music 3.1.

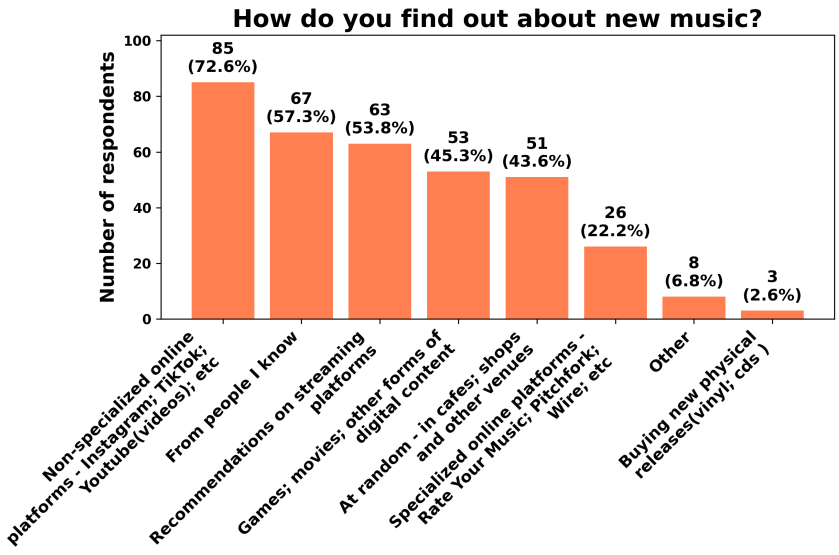

Figure 3.1: Music Discovery Methods Popularity

3.1 Streaming Services

As already illustrated in Figure 2.4, and further confirmed by the study of the International Federation of the Phonographic Industry [6], the

most common way of music consumption and discovery is through streaming services. Although many options are available, this section only considers those that are both popular and have unique features. The descriptions, rather than offering a general portrayal of each platform, will focus on the service-specific social features.

3.1.1 Spotify

One of the most prominent social features on Spotify is the "Spotify Jam" feature [7]. It lets people create a collective song queue that is then synchronized among all connected users. Moreover, volume, the order of songs, and other playback aspects can be controlled individually by each user.

Another notable feature is "Friend Activity" [8], the followers of a given user are currently listening to.

The "Listening Parties" feature also deserves a mention: these are live chats with a limited capacity that can be joined for a short time when new music is being released [9, 10]. This somewhat brings the offline collective music discovery experience to the online world.

During the implementation of this thesis (the text was initially written in spring–summer 2025), Spotify has also added support for user-to-user chats (from September 2025) [11]. This further supports the claim that social features are in demand on music platforms.

3.1.2 VK Music

VK Music is a music service integrated into the VK social network. Consequently, it is possible to send songs and playlists via private messages or add them to posts in groups. VK Music also provides an "Updates" tab, which is populated by the audio materials that any followed user or group has added to their "Liked Feed" [12].

3.1.3 SoundCloud

SoundCloud is one of the few platforms that lets its users leave comments and reactions on songs and playlists [13, 14]. In addition, each user has a feed consisting of personal uploads and reposts of others' content [15], which is visible when visiting their profile.

3.1.4 Bandcamp

Every Bandcamp user has a profile with four tabs: "collection", "wishlist", "followers", and "following". The most interesting feature is under the "following" tab: users can subscribe to available genres and then specify the ones they want to showcase to others viewing their profile. The "wishlist" tab is also noteworthy, as in this tab the users can showcase what they are looking forward to listening to in the future.

This concludes the streaming services section. Services that are focused on music, but not on the streaming itself, are discussed next.

3.2 Forums, Blogs, and Other Music Media

Different resources are gathered under these umbrella terms, from professional music review websites to amateur and personal pages. Again, only widely used services that offer something noteworthy are listed.

3.2.1 Rate Your Music

> "Rate Your Music is one of the largest music databases online. It is an incredible tool that can help you find and learn about new music to listen to." [16]

This website is one of the biggest platforms with user-created content. Every member can write reviews and set ratings for music releases, contribute to the "wiki" consisting of genres, thematic lists, and charts, and connect with other members of the forum.

3.2.2 last.fm

Last.fm is primarily a listening statistics and music discovery service. However, it also has a couple of notable social features:

1. A global and personal events pages. In the former, users are able to browse the upcoming events near their location, and in the latter they are able to showcase which events they plan to attend.

Other users can then see, either on the event page or on that user's profile, which events the user is interested in and/or plans to attend.

2. A "Neighbours" tab. The "Neighbours" tab shows other users with similar listening habits, based on shared artists and an overall compatibility score.

3. "Shouts" section. The "Shouts" section is a profile-scoped feed, where the users can leave comments and reply to what others have written.

This section has introduced a couple of possible features not previously seen in the listed streaming services, such as discussion forums and profile feeds.

The following section examines platforms that are not primarily focused on music but still influence how people engage with it.

3.3 General-Purpose Platforms

As mentioned earlier, one of the most interesting results of the survey is the high number of people discovering music on platforms that are not focused on music. Services like Instagram, YouTube, and TikTok, which are primarily used for sharing videos and other user-generated content, have become closely tied to how people find music and listen to it. In recent years, these platforms have started to promote music more directly by allowing users to embed tracks within their platform-native content.

Instagram, for example, added a special audio tool that lets users include music in their short videos ("Reels" [17]), helping songs reach more people through visual content.

YouTube automatically identifies songs used in videos and displays them in the description, making it easier to find and listen to them.

TikTok has had an even bigger impact: many songs have gone viral and later became hits in charts and streaming apps because people use them in popular videos (these are often called "TikTok Hits").

This shows that big platforms are not just reacting to users' interest in music, they actively help promote it by connecting songs with content that people want to watch and share.

An integration with such services can be a big plus for the platform. A possible feature could be a badge stating "This track was in a TikTok user Alice123 sent you today!".

Summary

The table 3.1 summarizes the comparison between the features mentioned in the sections above.
Difficulty suggests the possible complexity of implementation.
Mode assumes the typical context in which the feature operates.
Social Interaction Level indicates user involvement when using the feature.

Table 3.1: Comparison of Service-Specific Social Features

Feature	Interaction Level	Mode	Difficulty
Chat	Very High	Pair	High
Common Listening	High	Group	Very High
Forums/Wiki	High	Group	Medium
Personal Feed	High	Individual	Low
Song/Playlist Comments	High	Group	Low
Listening Parties	Medium	Group	Very High
Post Embeddings	Medium	Group	Medium
Friends' Listening Activity	Medium	Pair	Low
Following Genres Display	Low	Individual	Medium
Real-Life Events Section	Low	Group	High
Music Wishlist	Low	Individual	Low
External Services Integration	Low	Pair/Group	High

Overall, it is clear that numerous music-related platforms with social features built in are available across the Internet. However, services that focus on the actual audio delivery often support just a subset of the possible features. This, in turn, forces users to use multiple platforms in order to meet their needs. This project aims to bridge that gap, gathering multiple social features in one app.

Based on the research done in the previous and current chapters, the following social features were chosen for implementation, as they provide a good balance between the complexity of implementation and the level of social interaction:

- Chat.
- Personal/Global Feed.[1]
- Song/Playlist Comments.
- Friends' Listening Activity.

These and further requirements for the application are listed in the next chapter **Chapter 4**.

1. In comparison to `last.fm`, a common view with posts from multiple authors is going to be implemented as well.

4 Specification

In this chapter the general outline of the application is specified via functional and non-functional requirements. The individual requirements are constructed based on the 'MoSCoW' criteria [18]. "Must Have" requirements specify functionality without which the application would not be finished. Those are the main features and technical aspects that have to be implemented. "Should Have" requirements mostly represent features that would greatly improve the user experience, but are not strictly needed for the application to be fully working. "Could Have" requirements are reserved for possible future ideas which most probably would not be implemented in the current scope of the thesis.

4.1 Important Terms

Below is the list of important terms that appear throughout the requirements section and further chapters:

- **Anonymous User** – A User who is not yet registered in the system or has not logged in.
- **User** – An object containing information such as email, display name and other User parameters.
- **User Profile** – A Frontend-only abstraction which shows Base User information.
- **Artist** - A User with additional privileges, for example, Song or Collection creation.
- **Follower/Followee** - A relationship between a User and another User where one subscribes to another.
- **Friend** - A Followee and Follower who are mutually subscribed to each other.
- **Song** – An object representing a processed audio file with additional metadata such as title or creation date.

- **Collection** – A container that holds multiple Songs.
- **Playlist** – A Collection consisting of pre-existing (already uploaded) Songs.
- **Album** – A Collection consisting of newly uploaded Songs.
- **Playback** – A process during which the User is receiving or manipulating audio data.
- **Playback Item** – A Song or a Collection.
- **Playback Queue** – A User-specific list of Playback Items which determines the playing Song order.
- **Playback Device** – A device on which the app is currently used.
- **Friend Activity** – A display of the Songs currently listened to by the User's Friends.
- **Publication** – Textual User input.
- **Attachment** - Non-textual User input. For example, a Song attached to a Publication.
- **Reply** – The same as Publication, but created in relation to another, 'parent' Publication.
- **Feed** - A list of Publications accessible by any User. Either User-specific or Global (from multiple Users).
- **Chat** - A list of Publications specific to two or more Users.

4.2 Functional Requirements

Must Have

- The system shall support User and Artist models.
- The system shall allow Anonymous Users to create a User model and log in to the system using an email and a password.
- The system shall allow Users to change provided information (name, bio, avatar, etc.) after the model creation.
- The system shall support over-the-net audio Playback.
- The system shall provide a way to control the audio Playback. That includes shifting the Playback backward and forward, stopping and resuming it.
- The system shall provide a way to display Playback Items uploaded to it.
- The system shall limit content presentation based on a specific User. That includes Playback Items, User Profile, User Playback Devices, and other related items.
- The system shall support the creation of Playback Items. Playlists shall be created by all Users. Albums shall be created by Artists only.
- The system shall provide a Playback Queue and the ability to add or remove Playback Items from it.
- The system shall provide chat capabilities.
- The system shall provide a way for Users to add Publications (Comments) under Collections.
- The system shall provide a personal Publication system (Feed).
- The system shall provide a way for Users to create relations to other Users.
- The system shall have a way to search and filter Playback Items, Users, and Artists.

Should Have

- The system shall support adding Attachments, such as Songs or Collections, to Publications.
- The system shall provide a notification system. It shall display information about Users who followed the target User, new Reply Publication, etc.
- The system shall make it possible to filter Playback Items/Publications based on the User's relations to other Users.
- The system shall make it possible to add new Playback Devices and switch between them.

Could Have

- The system shall make it possible for multiple Users to listen to Songs in a synchronized manner.
- The system shall provide an Artist interface that allows Artists to see statistics related to their content.
- The system shall have a forum section.

4.3 Non-functional Requirements

Must Have

- The system shall provide access to its contents only to authorized Users. The only exceptions shall be the “Log In” and “Sign Up” pages.
- The system shall make unauthorized access to data as complex as possible, so that even in the case of misconfiguration of access rights, the data are hard to access.
- The system shall store uploaded content securely.
- The system shall store and transport sensitive User information safely. I.e., it should use safe transport protocols, such as HTTPS and WSS .
- The system should be resistant to the most common web-based attacks: CSRF, XSS, etc.
- The system shall, in case of unexpected failure, remain rendered on the screen and show the appropriate non-technical message to the User.
- The system shall render newly available information without User input. That includes updates to Publications, Playback Items, Playback Queue, and other related items.
- The system shall minimize the network load it has on the server and the User device. That includes reducing the data footprint and implementing eager loading strategies.

Should Have

- The system shall provide a responsive search method that reacts dynamically to User input.
- The system shall synchronize Playback across multiple devices.

Could Have

- The system shall
- The system shall support offline Playback.

The next chapter presents the high-level technology choices used in the implementation.

5 Implementation Planning

Before the actual implementation could begin, it was necessary to choose the languages, frameworks, and other technologies that would shape the overall architecture of the application. As mentioned before, the goal is a web application, hence, this chapter focuses on the technologies mostly suitable for web applications. This chapter discusses the following areas: Audio Processing and Playback, Authentication, Server–Client Communication, Backend, and Frontend.

5.1 Audio Processing and Playback

Audio Processing and Playback are the most important aspects of the application and were researched first. There are four main approaches used today: Basic Download, Progressive Download, Streaming, and Peer-to-Peer.

- **Basic Download**
  Basic Download works by downloading the entire file before Playback begins. It does so by making a standard HTTP GET request to which the server sends back an HTTP response with the media as raw binary data in its body. [19] Only when the full transfer has completed, the user can start the Playback.

  This method poses several limitations: for high-quality files, it typically requires significant storage space and time to download. Additionally, if the user's connection is unstable, it can make the application, which relies on playback, unusable. In cases of poor network conditions, the situation can be partially improved by providing multiple representations of the same file, such as a highly compressed version alongside a lossless one. However, after the download has started, the system is unable to adapt to the changing network state. In other words, the user is tied to the version of the file that has started to download and cannot benefit from, for instance, switching from a slow mobile network to a fast Wi-Fi connection.

- **Progressive Download** [1]
  Progressive Download still uses standard HTTP, but the client does not need to download the entire file before playback starts. Instead, it fetches a short initial part, begins playback, and continues downloading further parts as needed.

  This is usually implemented with HTTP byte-range requests. [21] To make this possible, the client relies on container metadata (e.g., the header of an MP4 file) that maps playback time to byte offsets. [22] With that mapping, the player can request the next few seconds of audio and also seek by requesting a different range.

  In comparison to Basic Download, since the file no longer needs to be downloaded fully, storage becomes less of an issue. The player only needs a small buffer in memory, and older data can be discarded once it has been played. The browser may still cache recently fetched ranges, but the application does not need to persist large portions of the file on disk.

  In addition, it becomes possible to approximate adaptive playback. One approach, presented by Färber Nikolaus [22], involves storing multiple representations as separate tracks inside a single container file (e.g., MP4) and switching between them during playback. The client then continues requesting the following parts of the file using HTTP byte ranges, but chooses the track that best matches the current network conditions. However, seamless switching requires the representations to be prepared in a compatible way, for example, corresponding parts of each track must cover the same playback time intervals and start at safe switching points.

  Segmented HTTP Streaming With Segmented HTTP Streaming, the audio file is first split into many small files (chunks), and a special manifest file is then generated that lists the chunks and describes which time range each chunk covers. [23] The client then downloads the individual chunks and uses them for playback instead of working with byte range requests for a single large file, as was the case with Progressive Download. Since the manifest provides a mapping of chunks to audio time slots, this

1. Sometimes called 'Pseudo-Streaming'. [20]

makes it possible to play the audio from any point and enables seeking.

Moreover, the original audio file can first be encoded into multiple formats of different sizes. Then, it is possible to preprocess those files and create a single manifest that allows choosing chunks of different sizes and quality for the same time slot. As a result, chunks can be selected based on network conditions or device capabilities, providing a more optimized playback.

Compared to Progressive Download, Streaming makes adaptive playback simpler, because the manifest provides a standardized way to describe multiple representations and their timing. It also simplifies retries and caching, since each chunk is fetched as a separate HTTP resource and can be handled efficiently by standard HTTP caches and CDNs. [24] Finally, the client logic becomes more straightforward: instead of computing byte ranges inside a large file, the player selects the next chunk from the manifest.

- **Peer-to-Peer**
  In P2P audio transfer, audio data is shared directly between users' devices rather than being served from a central server. Each user acts as both a client and a server, sending data to and consuming audio from other users in the peer-to-peer network. This method can reduce server load and improve access speed, but it relies heavily on user participation and network conditions.

Streaming was chosen as the playback method for this thesis.

Basic Download was excluded because it requires downloading the whole file before playback can start, making the usability of the application limited. Progressive Download improves startup time and seeking, but adapting to changing network conditions is not a standard part of the approach and would require custom preprocessing of audio files on the server side, as well as client logic that can utilize this approach. Peer-to-Peer transfer was also excluded, since it would rely on users' devices as sources of audio data, making playback speed dependent on peer availability and location.

Segmented HTTP Streaming addresses these issues by defining adaptive playback directly in the manifest and delivering the audio

as small chunk files that can be cached and retried like regular HTTP resources. Additionally, its integration into the service, as will be demonstrated later in **Chapter 6**, is straightforward.

The next key choice would be the authentication method, as it can significantly influence both the design of the Frontend and Backend parts.

5.2 Authentication

There are many different methods of authentication available, but only those suitable for web applications are considered: Basic Authentication, Session-Based Authentication, Token Authentication, and Third-Party Authentication (OAuth2).

- **Basic Authentication**
  Basic Authentication sends a username and password with each request in the HTTP `Authorization` header, typically Base64-encoded. [25]

  The browser then usually caches those credentials after the first successful login and automatically attaches the header to subsequent requests. [2]

  Nonetheless, it has multiple problems, even if used over HTTPS. Since Base64 is not an encryption algorithm but only an encoding, the header can be easily decoded back. And, since the Authorization header is sent with every request, it may appear in logs or telemetry, especially after HTTPS is terminated (e.g., after reaching the internal proxy).

  Moreover, there is no protocol-level notion of *logout* or per-device sessions. This means that either the Frontend must use workarounds that bypass this limitation, for instance, purposely sending a request with a manually configured `Authorization` header that has the wrong credentials, or the user has to clear their browser cache [26].

2. This is usually not clearly stated in the browser documentation. However, for example, in the case of Mozilla this is implied in bug tracker tickets and user documentation [26] [27].

- **Cookie-Based Session Authentication**
  Session authentication makes it possible to identify users without sending their credentials on every request. Initially, the user must log in with their credentials in the same manner as for Basic Authentication. But, instead of the browser caching and reusing the credentials, the server that processed the request creates a *session* record (a data structure holding relevant user information, such as ID, roles, etc.) and instructs the browser to set a cookie with the session ID. This ID is later sent on subsequent requests instead of the user credentials and is used to identify the logged-in user. [28]

  Compared to Basic Authentication, this enables server-side logout and revocation (by deleting or invalidating sessions), per-session security policies such as device binding and inactivity timeouts, centralized control of active logins, and a simpler implementation of features that depend on temporary state.

  The main trade-off is the need for a session store, typically implemented using a relational database or an in-memory key–value store, such as Redis [29]. This introduces additional operational complexity, infrastructure dependencies, and potential bottlenecks if not configured correctly.

- **Token Authentication**
  Token Authentication replaces raw credentials with a token that can be presented on each request instead of the login and password. The token is an opaque value issued by the server and typically sent in the `Authorization` header (usually under `Bearer` identifier).

  Unlike cookie-based sessions, the server does not look up session records. Instead, it validates the token on each request, for example, by checking a signed token with a shared secret or public key, and derives the user identity and permissions from the token itself. This removes the dependency on an external session store and can simplify scaling, as any instance that knows the verification key can accept requests.

  Token Authentication fits well for clients that are not browsers, such as command-line tools, mobile applications, and IoT de-

vices, where sending cookies is inconvenient. It is also commonly used for public APIs and distributed systems, where independent services need to authenticate to each other without sharing a central session store.

Nevertheless, Token Authentication has an important trade-off: tokens are hard to revoke. Once issued, the token becomes invalid only when its expiry time has elapsed. It is also possible to keep a *revocation list* and check every request against it, but that adds state, coordination, and latency. As a result, if an attacker obtains a valid token, they can usually use it until it expires.

- **Third-Party Authentication (OAuth2)**
  Third-Party Authentication delegates user login to a third-party identity provider (via the OAuth 2.0 protocol), such as Google, GitHub, and others. [30] Instead of handling passwords directly, the application trusts this provider to authenticate the user and assert their identity.

  In the standard authentication flow, the application redirects the user to the provider's endpoint, where the user logs in. The provider then redirects back with an authorization code, which the application exchanges for tokens that can be later used to obtain user information. Using that information, the application can then create internal user models with the provided identity.

  Thus, this approach offloads password storage, multi-factor authentication, and account recovery to the provider. However, it should not be the only authentication method the application provides, since it requires the user to have an account with one of the providers.

Based on the characteristics of the authentication methods, it was decided to use Session Authentication. Compared to Basic Authentication, it avoids sending user credentials with every request, supports explicit logout, per-session security policies, and extra session-related data. Compared to Token Authentication, sessions offer simpler revocation, do not require managing tokens, and are sufficient for a web application that does not need stateless or cross-service authentication. Third-Party Authentication based on OAuth 2.0 can be added on top of this design.

Another important concept that must be addressed is the server-client communication.

5.3 Server-Client Communication

Some of the features outlined in **Chapter 4** (such as immediate content updates, chat, and playback handling) require bi-directional communication. Hence, a method for communication from the server to the client must be implemented.

Because this is a web application, techniques such as Webhooks, gRPC, Pub/Sub, and others that are not directly supported by browsers are not discussed. Traditional Polling is also not considered due to its performance limitations and inefficiency. P2P techniques (e.g., WebRTC) are not listed because the application's architecture is client-server, not peer-to-peer.

This thesis focuses on four commonly used methods for server-client communication: Long Polling, Server-Sent Events, WebSockets, and Push API.

- **Long Polling**
  Long Polling is a technique where the client sends a request and waits until the server responds, holding the connection open until a response is received (or a timeout occurs). Once a response is received, the client immediately issues another request, therefore listening to server messages continuously. [31]
  While this method works in environments where server-to-client messages are rare, it is unsuitable for high-throughput scenarios (especially for frequent and small payloads), as it introduces a significant overhead by repeatedly opening and closing HTTP connections. [32]

- **Server-Sent Events (SSE)**
  SSE is a one-way communication protocol that allows the server to send data to the client over a single, long-lived HTTP connection.

  Firstly, the client establishes the connection via an HTTP request, to which the server answers with a response that has its *content type* set to `text/event-stream`. The server can then

reuse that open connection to send an unlimited number of responses (events). [33] Compared to Long Polling, SSE does not need to open a new HTTP connection for every update, thereby reducing the number of requests. At the same time, it is still based on plain HTTP, so it works in most browsers and fits well into networking setups.

However, this protocol also requires additional connection handling logic: the proxy and the server must keep track of all connected clients. Moreover, special handling on the Backend must be added in order to properly route messages. [34]

- **WebSockets**
  WebSockets, in comparison to SSE, provide full-duplex communication over a single TCP connection. This is achieved by firstly creating a standard HTTP connection and then upgrading it to the WebSocket protocol via the WebSocket Protocol Handshake. [35]. This method allows messages to be sent both ways at any moment and is optimal for applications that require frequent (i.e., multiple times per second) two-way communication. [32]

  However, adding WebSockets to an application also requires additional infrastructure and logic for managing connections and message routing. In addition, WebSocket messages do not follow the normal HTTP request-response model. They cannot be cached by standard HTTP caches [36] or CDNs, and they do not benefit from built-in features such as idempotent retries.

- **Push API**
  The Push API is a specialized web interface that allows a server to send messages to the browser without a preceding request from the client. [37] After the user grants permission, the browser creates a *push subscription* and exposes a special endpoint that the application server can use to send data to later without additional negotiation. [38] When such a request is received, the browser validates it and queues the message for processing by a special background script, known as a *service worker*. [39] Each domain (website) can register its own service worker, which is typically dormant and is only started when events such as push notifications need to be handled.

> While this mechanism is useful for notification systems, where short delivery time is not critical, it is not suitable for interactions that require an immediate response, such as updating the user's current playback state. Messages can be delayed inside the browser because multiple sites may have pending push messages to process, and the browser is free to schedule or throttle service worker execution and limit the resources available to it. As a result, push notifications typically have higher and less predictable latency compared to direct communication protocols, such as WebSockets. [37]

Based on the requirements outlined in **Chapter 4**, at least three features would require fast server-to-client communication:

- **Playback state synchronization.** The primary device would need to transmit playback updates to the server approximately once per second. The server would then propagate the synchronized playback position to all other active devices to ensure consistency.

- **Real-time typing indicators in chat.** The client would frequently send short updates to the server while the user is typing. The server would then relay these events to the other chat participant, displaying the current typing activity in real-time.

- **Immediate screen updates on newly created content.** The screen would need to update when, for example, new Publications are added on an open Feed. For this to feel responsive, the server will send the new content to the client without requiring prior new message requests.

The initial implementation used SSE, but it proved to be less suitable even with a small number of clients. That is why WebSockets are used instead, as they reduce the overhead of client-to-server communication for messages coming in rapid succession.

At this point, the core technologies that could influence the choice of languages and frameworks are known, and the Backend is discussed in the subsequent step.

5.4 Backend

5.4.1 Language

Three languages were considered for the Backend part of the application: Python, Java, and JavaScript (Node.js). All three can be used to build a web Backend, so the decision was mainly about how much time the implementation would take and how familiar the author is with the language and its tools.

Java offers strong static typing and higher performance compared to Python and JavaScript. [40] It also has mature frameworks such as Spring Boot and good tooling support in common IDEs. However, due to the language rules and syntax, it usually requires more boilerplate code and configuration for even simple endpoints. In addition, the author has limited experience with Java web development. Using Java would first require learning and testing the framework stack, which would reduce the time available for designing and evaluating the application itself.

JavaScript with Node.js would make it possible to use the same language on both the Frontend and the Backend, and it has good support for real-time communication while being the second fastest language among the three. [41, 40] It is also widely used for Backend services, with numerous libraries available for APIs and WebSocket-based real-time features, making it a good fit for this application in theory. Nevertheless, as with Java, the author has less experience with Node.js on the server side. Setting up a development stack, choosing and integrating the necessary libraries, and validating the design would again take a noticeable amount of time.

Python, in contrast, is the language the author knows best. It has mature web frameworks such as Django, FastAPI, and Flask, as well as good library support for common Backend tasks: authentication, database access, caching, background jobs, etc. While Python is slower in raw CPU benchmarks than Java or Node.js [41, 40], real-world web applications are usually limited by network and database latency rather than by pure CPU computational power. For an application that is primarily I/O-bound and uses external programs (i.e., processes) for CPU-bound tasks (such as audio processing), the performance provided by a Python Backend is sufficient and can be improved

further by adding more worker processes or server instances if needed. Therefore, choosing Python does not introduce a practical performance bottleneck for the expected load.

Taking these points into account, Python was chosen as the Backend language. The choice is primarily made due to the author's prior experience and the limited time frame of the thesis.

5.4.2 Python Framework Comparison

As mentioned previously, there are currently three widely used frameworks available for Python: FastAPI, Django, and Flask.

- **FastAPI**
  As the official website states:

  > 'FastAPI is a modern, fast (high-performance), web framework for building APIs with Python, based on standard Python type hints.' [42].

  Its main emphasis is compatibility with Python's `asyncio`, which makes it much easier to write asynchronous code in comparison to the other two frameworks. In addition, it integrates well with Pydantic [43] making input data validation straightforward. Moreover, routing/middleware does not require lots of configuration. [44]

  However, FastAPI is relatively new and simple. It can work well for small or highly custom systems, but for standard setups, it lacks many convenience wrappers. For example, the WebSockets integration does not provide connection lifecycle management or message broadcast capabilities. [45] Additionally, no predefined templates are available for basic CRUD operations; therefore, they must be implemented manually, which adds unnecessary complexity.

  And lastly, despite the steady growth of FastAPI usage, the community support is still not as wide as for Django or Flask. [46]

- **Django**
  Django is a 'batteries-included' [47] framework in the sense that

it has tools for the most common web development tasks. It provides advanced user session management, convenient CSRF and XSS protection integration, flexible built-in file storage abstractions, and integrated handling of media files (images, audio, etc.). Support for REST APIs can be added through `django-rest-framework` [48], which provides generic views, serializers, and routers that make the implementation of CRUD endpoints quick with minimal custom code on top. Real-time functionality can be implemented using `django-channels` [34], an official Django package that adds connection lifecycle management, group-based message broadcasting, and integration with Django's authentication and session systems. As the documentation states, one of the main advantages of this framework is the time savings it offers. [49]

Nonetheless, some notable drawbacks include its opinionated design choices, which can make it hard to add custom logic. Furthermore, the size of the resulting codebase can be larger because the framework ships with many modules that may remain unused, while still occupying storage space (for example, RSS integration or server-side rendering machinery). Finally, the most significant disadvantage is its limited support for asynchronous processing [50], which often requires the use of external task queues, such as Celery [51], for long-running or background operations.

- **Flask**
  Flask is also a lightweight Python web framework. [52] It provides only a small core with routing, request/response handling, and templating, while most other functionality is added through extensions. This modular design is suitable mainly for small services or prototypes with a limited and clearly defined scope.

  However, for typical JSON-based APIs, FastAPI offers similar functionality with fewer dependencies and less setup, while Flask usually requires several additional libraries and manual configuration to achieve the same result. At the same time, Flask, like Django, is based on the synchronous WSGI interface. WebSocket support is typically added via additional tools, such as

SocketIO [53], which requires the Frontend to use the same library instead of the standard WebSocket API, introducing unnecessary coupling. As a result, Flask effectively combines the disadvantages of the other two frameworks: it lacks Django's integrated ecosystem and does not provide FastAPI's modern asynchronous features. In the context of this application, it does not bring any clear advantage over Django or FastAPI.

Summing up, of the three frameworks, Django is the most suitable one. For a music streaming platform, its media handling capabilities, WebSocket support via `django-channels`, and CRUD endpoint abstractions of `django-rest-framework` lead to a better developer experience, shortening the overall implementation time. In addition, Instagram has proven that Django can handle big loads, hence, its WSGI reliance should not be a significant drawback. [54]

Having established the Backend framework, the next step was to choose the database.

5.4.3 Database

The Django framework supports several relational databases: SQLite, PostgreSQL, MySQL/MariaDB, and Oracle. Oracle is excluded due to its commercial license [55]. Otherwise, for a project of this size, any of these options would be fast enough, because the expected number of users and the amount of stored data are not significant. Therefore, performance is not the primary factor in choosing a database.

Instead, the choice is based on the available features and how well they are integrated with Django. In this application, two requirements are especially important: full-text search and UUID primary keys (see **Chapter 6** for details). And for these requirements, PostgreSQL is the most suitable option.

First, PostgreSQL offers good full-text search support and additional tools for fuzzy matching, such as trigram indexes. [56] As a result, it can be used as the base for full-text search instead of a separate service like Elastic or Sphinx.

Moreover, Django exposes these features through the `django.contrib.postgres` module, which simplifies the creation of full-text and trigram-based queries directly from Python code. [57, 58]

Second, PostgreSQL has native support for UUID values via a dedicated `uuid` column type. [59] This type stores UUIDs in a compact binary form and allows efficient indexing and comparison. [3] In contrast, in many other databases UUIDs are usually stored as plaintext (e.g., `CHAR(36)`), which is less space- and index-efficient. Django's `UUIDField` also maps cleanly to PostgreSQL's native type, so UUID-based identifiers can be used without extra configuration.

Lastly, with the database chosen, the Frontend application type, language, and framework had to be selected.

5.5 Frontend

5.5.1 Application Type

Modern web applications are typically built either around server-side rendering (SSR), where most HTML is generated on the server, or around client-side rendering (CSR), where most HTML is generated in the browser using JavaScript. A single-page application (SPA) is an application that keeps a single HTML document loaded and updates the user interface dynamically on the client, usually relying on CSR, sometimes combined with SSR for the initial page load.

In an SSR setup, frameworks such as Django generate most of the user interface on the server and send complete HTML pages (or partial updates) to the browser, often only requiring small pieces of JavaScript for interaction. [60] In contrast, an SPA typically fetches JSON or other data from the server and, based on it, renders and updates the user interface directly in the browser. [61]

The research for the Frontend began by deciding which of these approaches to use. For many web applications, rendering HTML on the server reduces client-side complexity, avoids duplicating logic between browser and Backend, and works reliably across a wide range of devices and browsers, making it a simpler and more robust default choice.

However, in this project, the application must maintain a lot of state directly in the browser: the current playback position, the playback queue, the volume level, the chat state, etc. These values change

3. If using time-based UUID types, e.g., UUIDv7

frequently and in small steps, sometimes several times per second, and require immediate feedback for the user. Moreover, the user interface must respond to events sent by the server over WebSockets, such as playback updates or chat messages. A purely server-rendered approach with full page reloads or round-trips for each small change would be unsuitable for this kind of highly interactive, real-time interface and would complicate state handling across devices.

Because of these requirements, the Frontend is implemented as a client-side rendered single-page application (SPA).

5.5.2 Language

Currently, there are many libraries and frameworks in different languages that can be used to write web interfaces; for example, it is possible to create them even in low-level languages, such as C [62] or Rust [63]. Nevertheless, JavaScript remains the industry standard because it is still the only general-purpose language directly supported by browsers. It means that JavaScript is the only language that can be executed natively by the browser's runtime, without the need for previous pre-processing (e.g., compilation to WebAssembly). This simplifies the development, build, and deployment process, and for these reasons, the client application is implemented in JavaScript.

5.5.3 JavaScript Framework Comparison

Modern JavaScript development is usually based on one of three libraries/frameworks: React, Vue, or Angular. [64] Each of them takes a different approach to structuring Frontend applications and has its own advantages and disadvantages.

- **React**
  React is a JavaScript library focused on building user interfaces through a component-based architecture. [65] It is defined by its declarative programming style, where the developer describes what the UI should look like for a given state, and React handles the updates. [66] Internally, React maintains a virtual representation of the DOM and only applies the minimal necessary changes to the real DOM, which helps keep many updates effi-

cient without the developer managing manual interface update operations.

One of the main strengths of React is its ecosystem. It has detailed documentation, numerous learning resources, and a large number of open-source packages. For instance, it integrates well with various state management tools and data-fetching libraries. Another advantage of React is React DevTools, which allows developers to inspect the currently rendered React elements and state directly in the browser, improving the debugging experience. [67] In addition, React is widely adopted and backed by a large community, so examples of real-world applications and solutions to common problems are easy to find. [68]

At the same time, React does not provide many features. It does not include routing, form handling, or data-fetching solutions in its core. As a result, these features must be added by the developer via separate libraries. This flexibility is useful, but it also means that the developer has to make more decisions and ensure that the chosen packages work well together.

Another point is performance optimization. For simple interfaces, React is usually fast, but in larger projects, "naive" code can cause many unnecessary re-renders, leading to average performance compared to other frameworks. [69]

- **Vue**
  Vue, in comparison to React, is a framework: it includes many standard features out of the box, such as routing and form handling. [70] It offers a template-based syntax and single-file components, which combine markup, logic, and styles in a single file. This structure can make it straightforward to start new projects, since less setup is needed.

  The main drawback of Vue is its adoption compared to React and Angular. [64] While it has an active community, there are fewer third-party packages, fewer example projects, and fewer guides for advanced use cases. In the context of this thesis, where, as mentioned earlier, development time is a key factor, this can mean additional work, because features that exist as mature

packages in other ecosystems may need to be implemented manually or with less proven tools.

- **Angular**
  Angular is a full-stack framework for building web applications. [71] Like Vue, it provides many tools out of the box, including routing, form handling, and a dependency injection system. It is built on top of TypeScript and uses strongly typed code and clearly defined architectural patterns. This can lead to more structured and robust code in large projects with many developers.

  In addition, Angular provides built-in tooling and conventions for project setup, templates, and asynchronous data handling. This makes it easier to follow a single, consistent approach across the codebase instead of combining multiple separate libraries.

  As can be seen, Angular is aimed towards bigger, enterprise-oriented projects. While they can benefit from the structure and order that Angular brings, in smaller applications, the extra code required to create and maintain the project can be a significant disadvantage, especially in the case of this thesis.

While all three frameworks offer modern development patterns and are suitable for production, React was chosen as the Frontend framework for this project. It is mature, widely used, and has an extensive ecosystem of libraries for routing, state management, and UI components. Its small core does not impose a strict application structure, which is convenient in smaller projects. Together with the available debugging tools, this makes it possible to iterate on the user interface quickly and to integrate real-time features such as playback synchronization in a straightforward way.

5.6 Summary

The platform follows the Model–View–Controller architectural pattern [72] and is organized into three logical layers:

- **Model**
  The Model layer is responsible for representing and manipulat-

ing the application data. In this project, it consists mainly of the Django models with entities such as users, songs, playlists; and the data processing logic, for example audio conversion or data querying.

- **View**
  The View layer is what the user interacts with directly. It presents the data from the Model and turns user actions into requests that can be processed by the rest of the system. Here, the View is implemented as a single-page application in JavaScript using React. It renders the interface, displays information such as playback state or friends' activity, and sends user actions (for example, play/pause commands or chat messages) to the Controller.

- **Controller**
  The Controller layer connects the View with the Model. It receives HTTP and WebSocket requests from the Frontend, applies cookie-based session authentication and access control, and then calls the appropriate Model logic. In this platform, the Controller corresponds to the Django REST API and WebSocket endpoints.

In other words, the Django models and database layer form the Model, the React single-page application acts as the View, and the Django-based REST and WebSocket endpoints play the role of the Controller that coordinates communication between them.

The next chapter, **Chapter 6**, describes the concrete implementation of these parts and how they work together to form the complete music streaming platform.

6 Implementation

A high-level overview of the entire system can be seen in the following diagram 6.1.

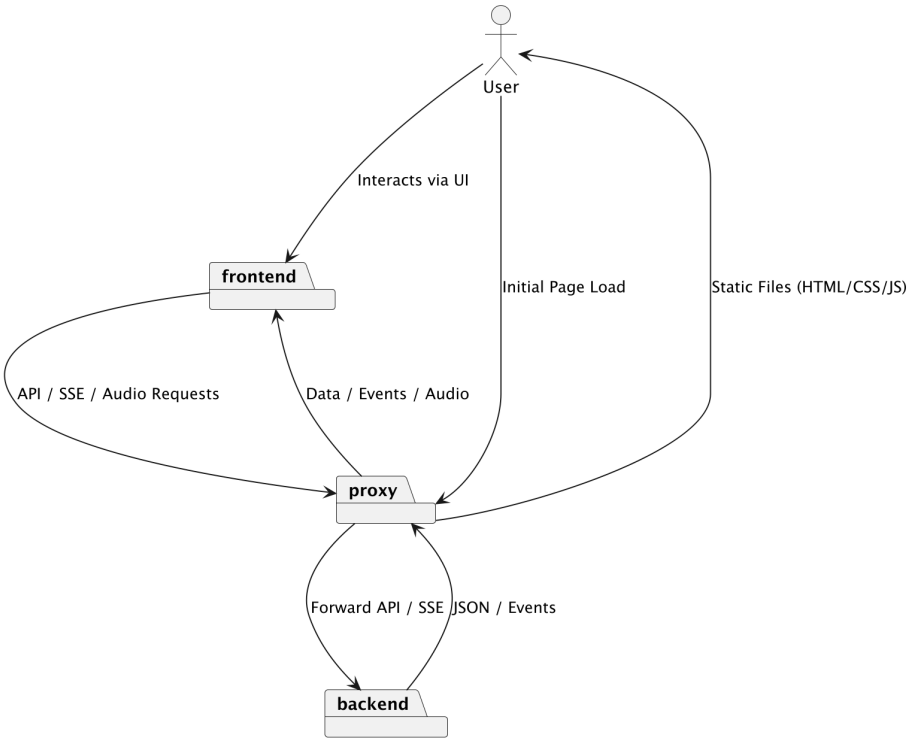

Figure 6.1: System Overview

The detailed Backend and Frontend diagrams are provided in the relevant sections.

6.1 Audio Processing and Streaming

This section provides the core principles of Audio Processing and Streaming.

As was stated in the **Chapter 5** Segmented HTTP Streaming is going to be used as the main method of audio presentation; therefore the Streaming Protocols are discussed first. Then, the stages that are

needed to make an audio file streamable are described: Transcoding and Packaging. These stages are presented in the reverse order, as Packaging sets requirements that affect Transcoding. In the end of the section, the software tools used for these processing steps are selected.

6.1.1 Streaming Protocols

Two protocols were chosen for audio streaming: MPEG-DASH[73] and HLS[74]. The basic pricnipals behind them were introduced in the Segmented HTTP Streaming section.

At first, only DASH was considered, because it works better in varying network conditions [75], and is not limited to only a select amount of audio codecs. [23]

However, DASH support in audio players is typically implemented via browser Media Source Extensions[76], which iOS does not support.[77] Since iOS accounts for roughly 15% of all operating systems worldwide [78], HLS support had to be added as well.

Both streaming protocols will therefore be used: DASH on devices that support MSE, and HLS on iOS. HLS licencing will not be an issue, since it is free to use for non-commercial purposes.

With the protocols selected, the audio representation required for them to work is discussed next.

6.1.2 Packaging

Packaging is the step where the encoded audio is prepared for Streaming. [79] It mainly includes dividing the audio into chunks and generating manifest files that reference them (an `*.mpd` file for DASH and `*.m3u8` files for HLS).

Because the platform serves both DASH and HLS, it is useful to generate only one set of chunks and reuse it for both streaming methods. For this reason, the packaged output follows Common Media Application Format (CMAF)[1] ,which defines a common way to store chunks as fragmented MP4 (fMP4) containers. [80]

The resulting output would contain one initialization segment (`*_init.mp4`), which stores audio metadata and decoder setup, and a

1. This is usually not implied strictly by packager utilities, but can be observer by running, e.g., 'ffprobe' CLI utility on the converted file.

sequence of media segments (`*.m4s`), which store the audio data in short time intervals. In addition, this approach is required for streaming FLAC encoded audio via HLS. [81]

But before the audio is packaged, it is needed to be converted to multiple formats in order to support bitrate switching. The next subsection discusses this process, which is called Transcoding.

6.1.3 Transcoding

Transcoding is the process of decoding audio data and re-encoding it, typically to a different codec and/or different encoding settings: bitrate, sample rate, channels, etc.

Usually, when high-quality audio is recorded, it is initially saved in raw audio formats, such as WAV, which results in big file sizes which are impractical for Streaming.[2] To reduce the size, the audio is usually processes using a special program called *codec*, which is a program that compresses and stores audio in a more efficient format.

Lossless codecs, such as FLAC [82], keep all audio information, so the audio sounds exactly the same as the original raw data. In contrast, lossy codecs (MP3 [83], AAC [84], OPUS [85], etc.) remove some information to reduce file size, which can slightly reduce quality, especially at low bitrates.

In this implementation, the lossless FLAC codec and lossy AAC and HE-AAC codecs were selected due to their good performance in web-based streaming scenarios. [86, 87, 88]: Other codecs that theoretically promise to provide better sound quality at smaller file sizes, such as Opus or MP3 , were researched; however, for higher bitrates that are used in Streaming they did not provide any advantage. Moreover, only *-AAC and FLAC codecs are supported for HLS streaming. [89]

Codecs usually provide several parameters that affect the output, such as sample rate, the number of channels, and bitrate. Bitrate describes how much audio data is stored or transmitted per second and is typically expressed in KB/s. It is commonly used because it provides a direct way to control the trade-off between audio quality and file size.

---

2. For instance, the resulting WAV file for a 3-minute stereo audio recording with a bit depth of 24 bits and a sample rate of 41khz is approximately 50 MB.

Before selecting the encoding configuration, it is also important to distinguish between two encoding strategies: Constant Bitrate (CBR) [90] and Variable Bitrate (VBR). [91]

CBR keeps the bitrate fixed across the entire audio file, resulting in predictable chunk sizes and smooth playback during Streaming. VBR adjusts the bitrate depending on content complexity, which can provide better quality at a smaller size compared to CBR but can lead to inconsistent segment sizes, making Streaming less stable [92].

For this reason, a **multi-CBR setup** was selected. While less efficient in terms of quality-to-size ratio compared to VBR, CBR provides greater reliability and predictability when Streaming.

The following configuration was chosen based on the available research and recommendations [86, 87, 88, 93, 89, 94]:

- **AAC**: 96, 160, and 320 kbps
- **AAC-HEv2**: 24 kbps
- **FLAC**: lossless

As mentioned before, multiple bitrates are included in order to support adaptive streaming.

The tools which are used to do the Packaging and Transcoding are introduced next.

6.1.4 Processing Tools

Transcoding is done via `ffmpeg` [95], which is an open-source CLI tool. It is used because it offers support for all of the selected codecs, can be run programmatically as a script and provides high speeds of audio conversion, since it is written in C and is optimized to use multiple threads.

`ffmpeg` provides multiple codec implementations, out of which `libfdk_aac` and `flac` were used. `libfdk_aac` was chosen because per documentation it is the best AAC codec that `ffmpeg` provides, and only a single implementation for FLAC encoding exists. [96, 97].

The resulting `ffmpeg` command that is used for Transcoding looks like this:

```
ffmpeg -i *input file path*
```

```
-vn -sn -dn
-c:a *encoder* *encoder extras*
-b:a *bitrate*
-movflags
+faststart
*resulting file path*
```

Where individual flags are:

- `-i *input file path*` selects the input media file.
- `-vn` disables video streams.
- `-sn` disables subtitle streams.
- `-dn` disables data streams.
- `-c:a *encoder*` selects the audio encoder/codec used for the output (e.g., `flac`, `libfdk_aac`).
- *encoder extras* are additional encoder-specific options (e.g., `-profile aac_he_v2` for `libfdk_aac`).
- `-b:a *bitrate*` sets the target audio bitrate (e.g., `96k`).
- `-movflags +faststart` moves the MP4 metadata (`moov` atom [98]) to the beginning of the file to improve download/streaming startup.
- `*resulting file path*` sets the output file path.

Even though `ffmpeg` is able to do the Packaging step as well, a dedicated packager tool was decided to be used. This is due to the problems that `ffmpeg` has with generating manifests that are compatible with Shaka Player [99] (which is used on the Frontend for audio playback, see Frontend for details)

Therefore, Shaka Packager was selected for the packaging step. [100] It supports both DASH and HLS, can generate CMAF-style fMP4 segments, and can produce both manifests in one run. In addition, it integrates well with Shaka Player.

The resulting Shaka Packager command that is used for Packaging looks like this:

```
packager
  "input=*file path 1*,stream=audio,
   init_segment=*base path*/audio_0_init.mp4,
   segment_template=*base path*/audio_0_$Number$.m4s,
   playlist_name=audio_0.m3u8"
  "input=*file path 2*,stream=audio,
   init_segment=*base path*/audio_1_init.mp4,
   segment_template=*base path*/audio_1_$Number$.m4s,
   playlist_name=audio_1.m3u8"
  ...
  --segment_duration 3
  --generate_static_live_mpd
  --mpd_output=*base path*/manifest.mpd
  --hls_master_playlist_output=*base path*/master.m3u8
```

Where individual flags are:

- `packager` – the Shaka Packager binary.
- `"input=*file path N*,stream=audio,init_segment=*path*, segment_template=*path*,playlist_name=*name*"` defines one packaging input, where:
  - `input=*path*` is the local path to the downloaded source audio file.
  - `stream=audio` selects the audio stream from the file for packaging.
  - `init_segment=*path*` sets the output path for the CMAF/fMP4 initialization segment (`*_init.mp4`).
  - `segment_template=*path*` sets the template for media segment file names.
  - `playlist_name=*name*` sets the name of the per-representation HLS media playlist (variant playlist), e.g., `audio_0.m3u8`.
- `-segment_duration 3` sets the target segment duration in seconds.
- `-generate_static_live_mpd` generates a static MPD with, which is used for better compatibility with audio players.

- `-mpd_output=*path*` sets the output path for the DASH manifest (`manifest.mpd`).
- `-hls_master_playlist_output=*path*` sets the output path for the HLS master playlist (`master.m3u8`).

Having determined how the audio is going to be processed and represented, the data and API model implementation is discussed in the further section.

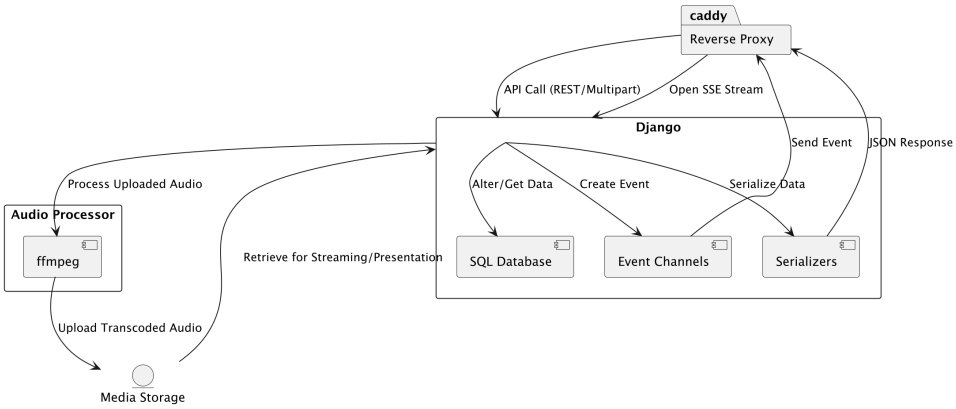

Figure 6.2: Backend

6.2 Backend and API

The Backend and API description is organized by the individual folders under the `musikk/src/backend/musikk` directory.

Most of the subsections are comprised of the following parts:

1. **Models** – Objects that define the data schema using Django's ORM.
2. **Serializers** – Code that converts Django models or native Python types into formats suitable for HTTP transmission (and vice versa).
3. **Views** – Handlers for HTTP requests that provide the relevant business logic and return appropriate responses.

In order to better understand the overall Backend data model and class composition, a full class diagram can be seen first. 6.3. It shows attributes, methods, and relationships of classes. No separate ERD diagram is provided, as Django's models, which define the data structure, are encapsulated into the class diagram.

6.2.1 Directory `base`

This app defines the abstract `BaseModel` class from which all other models inherit.

It has three attributes: `uuid, date_added, date_modified`. Date attributes are set on model creation and modification. `uuid` has to be explained. Since all the data fetching in Frontend will happen via the REST API, a unique resource identifier is needed. The drawback of using an `ID` of the model record, which is added by default for all models by Django, is the possibility of enumeration attacks **??** and unauthorized scraping of the web page. It is much easier to get an existing ID and simply send requests to API endpoints with an incremented ID value than to guess a random 128-bit string.

A retrieve serializer for the `BaseModel` is defined as well, it is also the base class for the serializers of other models.

6.2.2 Directory `api`

Firstly, it is necessary to say that the resulting API follows REST [101] practices; it only deviates from them when strictly following the rules would degrade the readability and usability of the API. This can happen, for example, when an endpoint is used to execute an action instead of a data operation. The other rule that is not followed is the presence of hypermedia links on related resources, as in the current architecture, it makes more sense to use individual model identifiers and construct URLs from them instead.

The API itself is versioned, with the current version being *v1*. In the case of the following apps, all the API-related files are always defined under the `api/v1` folder.

As for the endpoint, all views that operate on data in some way require authentication, which can be determined by the presence of `permission_classes = [IsAuthenticated]`

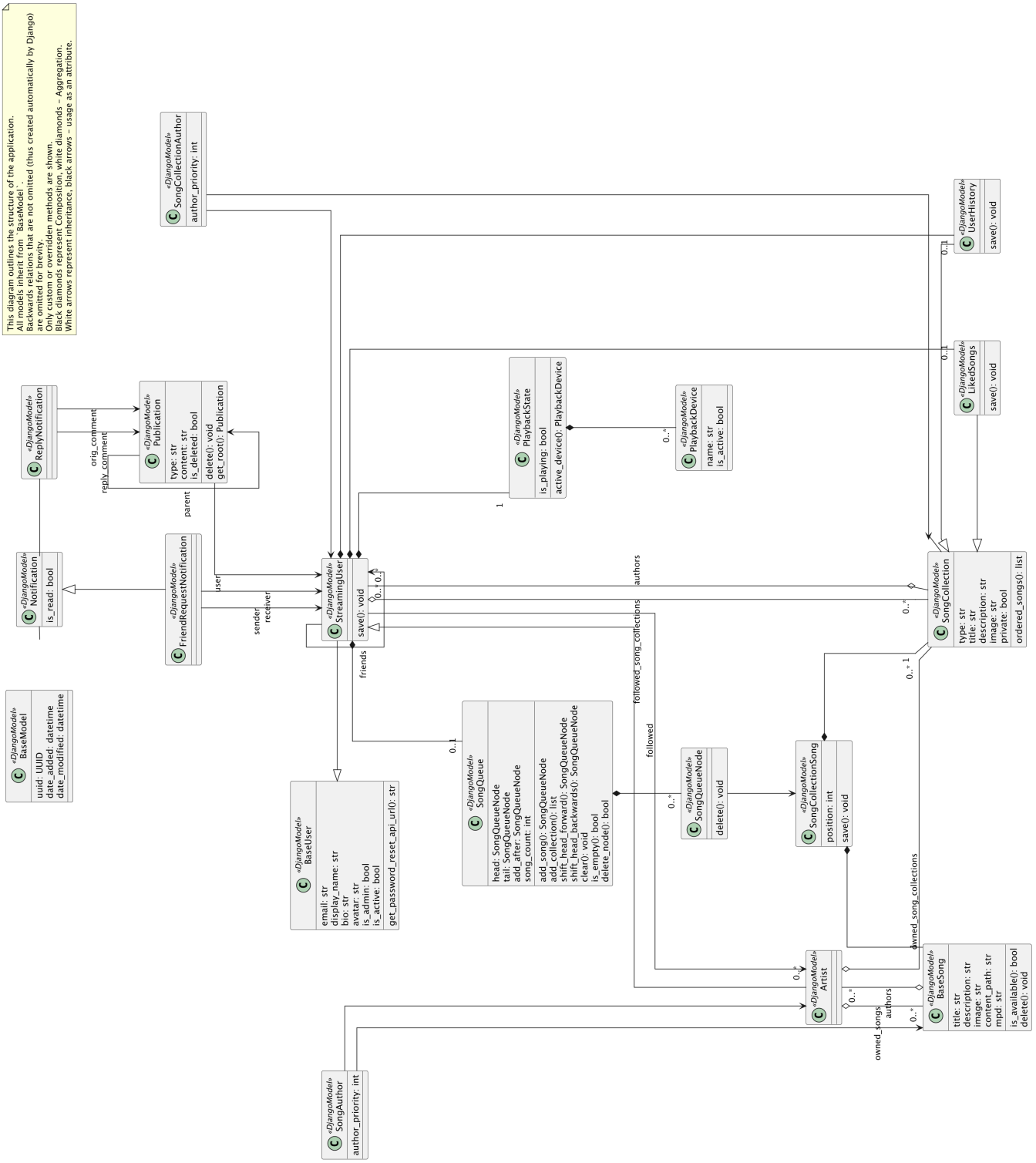

Figure 6.3: Backend Class Diagram

attribute on the view. User roles are also differentiated by checking the existence of the appropriate model, e.g., for Song creation, the User set on request has to have a corresponding `Artist` instance.

6.2.3 Directory `users`

User classes and token logic are defined in this app.

There are three main User classes:

- `BaseUser` - declares common fields for all other models, such as `email, display_name` and others.
- `StreamingUser` - the main class representing an account with privileges to use the streaming platform.
  Additional attributes are declared for it, namely:
  - Relations to other users: `friends`, which are other `StreamingUsers`,
    friend requests of whom the user has accepted (or the other way around);
    `followed` - the `Artists` to the updates to which the User has subscribed.
  - User-specific Song Collections: `history` containing all listened Songs,
    `liked_songs` containing favourited Songs,
    and `followed_song_collections` - the Collections which the user has saved to their profile.
  - `song_queue` - a special structure containing already/to be listened to Songs of the User,
    it will be described later in the dedicated app.
- `Artist`: a subclass of `StreamingUser` with additional capabilities, such as Song uploads.

JWTTokens are also customized in this app. They are based on the implementation from `djangorestframework-simplejwt` [**simplejwt**]. The token system follows the standard logic of access and refresh tokens: the access token is short-lived and used to authenticate regular

API requests, while the refresh token is long-lived and can be used to obtain new access tokens without requiring the user to re-authenticate.

Authentication for each HTTP request involves attaching the access token as an HTTP *Bearer* header. After parsing the request, the token is retrieved and validated. After validation, the User identifier is extracted and used to identify the corresponding User record. User validation logic is changed so it uses User's `uuid`, hence it is added to the token payload. This is done for the same reasons outlined in the Directory `base`.

Basic serializers for all User models are declared, allowing for retrieving information about specific Users or their creations and updates.

The following views are added as well:

- `UserRetrieveUpdateView`
- `UserCreateView`
- `UserFriendsView`
- `UserFollowedView`
- `UserFriendsCreateDeleteView`
- `ArtistFollowersView`
- `UserEventViewSet`

The purpose of all views, except for the last one, is to send the information about Users or perform create/update actions on the related data.

UserEventViewSet is a special view used to implement the SSE connection, the details are provided in the subsection *sse*.

Views for token generation and renewal have also been declared. Their respective endpoints are:

- `/api/v1/token/` — accepts user credentials (email and password) and returns a token pair (access and refresh tokens).
- `/api/v1/token/refresh/` — accepts a refresh token and returns a new access token.

6.2.4 *sse*

As was mentioned in Chapter Could Have, `django-eventstream` package is used for the Server Sent Events configuration. Internally, it creates special objects called *channels* to which clients can subscribe via an HTTP request. Then, when a new event for a specific channel is created, e.g., via the `django-eventstream.send_event()` method, all clients subscribed to that channel will receive an HTTP response with the specified content.

'UserEventViewSet' listens for incoming SSE connection requests from Users and creates a personalized channel using the User's UUID.

Events are used throughout the system primarily to signify to the Frontend that some data is stale. An example of this is the aforementioned 'UserFriendsView', the post request handler of which is defined like this:

```
class UserFriendsCreateDeleteView(APIView):
...
def post(self, request, *args, **kwargs):
    ...
    user.friends.add(sender)
    Event.invalidate_event(
        EventChannels.user_events(user.uuid),
        ["user", "friends", str(user.uuid)]
    )
    ...
```

Here, the custom `invalidate_event` signifies to the Frontend that the `friends` list has changed and should be refetched.

The *invalidate* logic is present for views operating on models used in the Frontend, specifically in view handlers which change data. That way, the client will always have fresh information.

Event re-delivery is also handled by the package set with this parameter in Django settings:

```
EVENTSTREAM_STORAGE_CLASS = \
    'django_eventstream.storage.DjangoModelStorage'
```

This tells the package to store the events in the database and in case of network failure or client disconnect they are redelivered.

Since the SSE part has been explained, it is safe to consider the main part of the application – the *streaming* package.

6.2.5 *streaming*

This app defines the main models responsible for the audio content and playback handling.

Songs and Song Collections

The backbone of the application is the `BaseSong` and `SongCollection` models, which are declared in `streaming/songs.py` and `streaming/song_collections.py` files.

- `BaseSong` contains the metadata about each individual song - `title`, `description` and `image`.

  Then, there are attributes that contain the information about the underlying song file: - `content_path` representing the root directory with Song files(chunks, manifests) - `mpd, m3u8` providing manifest paths for DASH and HLS, respectively.

  The last attribute is `draft`. It is used in the Song creation process, which will be explained in the view description.

- `SongCollection` represents a container for Songs and can be of two types(set with `type` attribute) – either a Playlist or an Album. A Playlist is a collection formed from existing Songs, which both StreamingUsers and Artists can create. An Album, on the other hand, is a collection that can be created only by an Artist containing unique, uploaded Songs.

  Because an individual `BaseSong` can be in multiple collections, a Many-to-Many relation is defined between `BaseSong` and `SongCollection` named `SongCollectionSong`. A `position` attribute is added to enable the arbitrary ordering of the songs.

  Another attribute on the `SongCollection` is `comments`. As the name suggests, these are pieces of text that each User can add to the collection. The underlying model is discussed in the *social*.

  The same metadata as in `BaseSong` can also be set. However, there is one extra attribute - `private`. It is used to signify which collections should not appear, e.g., in global search results.

In addition, there are two subclasses of `SongCollection` - `UserHistory` and `LikedSongs`. They are moved to separate classes because, in the future, they will most probably have custom logic added to them. In the app's current version, only the ordering that makes the newest added songs appear first is defined.

Each `SongCollection` and `BaseSong` also has a relation to the Users that created it(and visa versa). It is defined as an M2M relationship with an intermediary table - `SongCollectionAuthor` and `SongAuthor`. Attribute `author_priority` sets the order in which the authors should be returned from the API and displayed on the Frontend.

As for the views and endpoints - a standard set of CRUD operations is implemented for both models. Additionally, there are special views for User interactions with models, such as `SongAddLikedView`, which adds the provided Song to User's `LikedSongs` or `SongCollectionRemoveSong`, which lets the User remove a Song from a collection, if that User created it.

However, some things have to be said about Song and Collection creation. The design choice is not to allow users to create isolated Song instances; if the User wants to upload Songs separately from a Collection, a single-song Album would be created instead. This greatly simplifies the logic when handling User playback interactions later on. For example, it makes it possible to use the aforementioned `SongCollectionSong` instead of `BaseSong` in most views. As this relation contains the information about both models, we can, for instance, accurately show not only which Song is playing, but a Collection as well, without tracking both models. This has further benefits, which can be seen in the Frontend implementation.
Apart from that, the Collection creation process is multi-stage, and two views are responsible for it: `SongCreateView` and `SongCollectionCreateView`. Whenever a User wants to create a new Collection, firstly the Songs are processed. If an error happens during this stage, the response with successful and failed Songs is returned, and `draft` attribute is set to *True* for the created Songs. This is done because the User can abort the process of creation, e.g., by closing the whole browser application, and there would be no proper way to signal what Songs should be deleted. A simple crontab job or `celery` library could be utilized to run periodic tasks which would clean up Songs in the

draft state.
When creating a Song, multiple validation stages are made for the User input, both on the Frontend and the Backend. For instance, `ffmpeg_wrapper.py` checks the audio file size, its mime type via [102] library, and the underlying format, with available formats specified in `ffmpeg_wrapper.ALLOWED_FILE_TYPES` variable.

Having discussed the main building blocks of the audio content part of the application, `SongQueue`, which operates on these models, can be presented.

SongQueue

`SongQueue` represents a list of previously listened Songs and the ones that are going to be listened by the User.

The choice of the underlying structure was made based on the desired operations. Basic addition and removal from the queue had to be supported, as well as the differentiation between playing a Song/-Collection or adding it to the queue. For example, when a user clicks play on a Song, it has to become the new "head" of the queue. In case of clicking "add to queue" this Song would be added after other items that have been queued previously. This implies that addition/removal can happen in any part of the list. As a result, a Doubly Linked List pattern has been chosen for this model.

As the queue information had to be persisted in the database, a common implementation with nodes that point(using foreign keys) to the next and previous node might not have been the most efficient. An interesting approach proposed online was to use a *position* column instead of foreign key references. That way it would be possible to retrieve the whole queue with an SQL statement like this:

```
SELECT *
FROM (join node table, queue table, and StreamingUser table
      to find the queue nodes of a particular User)
WHERE node.position > (join queue and nodes to find the head of the queue)
ORDER BY node.position
```

While using a `position` column simplifies queue traversal and ordering, it introduces potential issues when it comes to frequent inserts and deletes.

Every time a node is inserted between two existing positions, a new `position` value must be chosen such that the ordering remains correct. If positions are represented by integers, repeated insertions between tightly packed values (e.g., inserting between positions 1 and 2) will eventually require re-indexing the entire list to free up space. This operation can be expensive in terms of performance. Similar problems can occur during deletes as well.

These limitations could be mitigated, for example, by using high-precision floating point numbers, powers of numbers or large increments like 1000 to delay the need for full re-indexing. However, without the data showing the expected usage of the queue, it is hard to tell whether this approach would be beneficial.

That is why a more straightforward approach with identifiers of neighboring nodes was chosen, at least for now.

Two models comprise the queue functionality - `SongQueueNode` and `SongQueue` itself.

Each node carries information about the `SongCollectionSong` attached to it, as well as links to the previous and next node.

`SongQueue` has foreign keys to the nodes which signify the `head` and `tail` of the queue. Moreover, a special, third attribute is declared - `add_after`. This is also a foreign key to one of the nodes, but this time, it indicates the node after which items should be added when the "add to queue" action is selected. I.e. if the user had selected "play" on a Song, it would become the queue head. Then, if the user selected "add to queue" on five different Songs, then the head would still point to the playing Song, but `add_after` would contain the key of the fifth Song added using the second action.

Multiple operations are supported, with the main ones being:

- Setting the queue head. In the case of Collections, e.g., when the User clicks "play" on a Collection object, the entire Collection is added to the queue.
- Shifting the queue head backward and forward.
- Removing a node.
- Clearing the entire queue.

The final aspect that needs to be addressed is managing the playback state.

Playback State and Playback Devices

The term "playback" represents the state of the sound, whether it is being played in a given moment. Each User has an attribute named `playback_state` represented by the `PlaybackState` model. The playback can be toggled by using the `PlaybackState.is_playing` attribute.

Because the User can open multiple instances of an application, the playback handling can not be unloaded to the Frontend only, as it would not be synchronized across the application instances.

As a result, the application has to differentiate the User's "sessions", which is done via the `PlaybackDevice` model. It represents the individual devices on which the application is currently open. Each device has `is_active` attribute, signifying where the sound should be played and where to display controls(seek bar, volume).

It would be expected for all instances of the application to show the same playback state(playing/stopped), and intuitively clicking the play button would have to result in playback being toggled on the active device with visual confirmation on all other devices. In addition, the active device must be able to be switched, which is done by the appropriate `PlaybackDeviceActivateView` handler.

The standard serializers and views allowing state/device manipulation are implemented. However, `PlaybackDeviceDeleteView` deviates from the standard rules, as it allows non-authenticated requests and softens the CSRF restrictions. This is done due to browser limitations, shown in the **??**. The authentication is still made "by hand" inside the handler itself.

This concludes the audio part of the application.

6.2.6 *social*

Before examining the contents of the *social* app, an important Django concept has to be brought up – the `ContentType` framework and `Generic Relation` [103].

The `ContentType` framework provides a way to refer to any model in the system using a generic identifier. `GenericRelation` uses the `Con-`

tentType framework by storing the target model's type and primary key in two separate fields: a foreign key to the ContentType model and an object ID. Together, these allow a single relation to point to any instance of any model, enabling polymorphic associations without hardcoding specific model references.

This concept is used to implement the Publication model.

For a better understanding, the model code can be found below. Here, the content_object field is a GenericForeignKey that dynamically joins the content_type and object_id fields to establish a reference to any model instance. This allows each Publication instance to be associated with an arbitrary object, such as a Song or a Collection, without requiring a fixed foreign key to a specific model.

```
class Publication(BaseModel):
    class PublicationType(models.TextChoices):
        COMMENT = "comment", "Comment"
        POST = "post", "Post"

    RELATED_MODEL_TYPE_MAP = {
        "user": "users.StreamingUser",
        "collection": "streaming.SongCollection",
        "song": "streaming.SongCollectionSong",
    }

    type = models.CharField(choices=PublicationType.choices)
    content = models.CharField(max_length=255)
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL, null=True,
        on_delete=models.SET_NULL
    )
    parent = models.ForeignKey(
        "self", null=True, blank=True,
        related_name="replies", on_delete=models.SET_NULL
    )
    is_deleted = models.BooleanField(default=False)

    content_type = models.ForeignKey(
        ContentType, null=True,
        blank=True, on_delete=on_content_type_delete
```

```
    )
    object_id = models.PositiveIntegerField(null=True, blank=True)
    content_object = GenericForeignKey()
```

Another attribute that needs an explanation is `type`. Currently, it can have one of two values - `comment` and `post`. The comment type is used for comments under audio content(collections), while `posts` represent standalone User publications.

The `Publication` model also contains a `parent` attribute, allowing for building trees of publications(i.e., replies).

Standard CRUD serializers and views for publications are provided, with the addition of custom handling of the related models.

Views related to the publication creation also have a special handling of instances that are considered as replies. They create a special `ReplyNotification` object, discussed in the next subsection *notifications*.

6.2.7 *notifications*

In order to notify users about actions such as replies to their publications or new friend requests, a notification system was implemented.

The following two notification types are currently supported:

- `ReplyNotification` – created when a user replies to a publication (e.g., comment). It stores references to both the original and reply comments. The corresponding manager triggers an SSE event to the original author's channel, making Frontend refresh the notifications list.

- `FriendRequestNotification` – created when a friend request is sent. It stores references to both the sender and the receiver. Upon creation, same as for `ReplyNotification` an SSE event is emitted to the receiver's channel.

Both classes inherit from `Notification` class. It has a single attribute – `is_read` which helps to differentiate between already seen and new notifications on the Frontend

6.2.8 *recommendations*

The *recommendations* app currently provides basic search functionality across Songs, Collections and Users using Django's native PostgreSQL full-text support.

The core search logic is implemented in the `SearchView`, which uses `TrigramSimilarity` to retrieve fuzzy matches based on string similarity. Trigram search is appropriate for this purpose because many of the search targets, such as usernames or collection titles, often contain informal, abbreviated or non-standard text. In such cases, traditional vector-based full-text search (e.g., via `SearchVector`) would perform worse, as it is optimized for longer, more "text-like" content.

A dedicated search engine like *Elasticsearch* was not introduced for two reasons:

1. The scale of the platform does not currently justify the additional infrastructure, deployment, and synchronization complexity that comes with search engines.

2. PostgreSQL's trigram extension provides sufficient performance and accuracy for the expected use case. Moreover, Django offers first-class support for it through the `django.contrib.postgres.search` module.

This design ensures efficient search behavior while avoiding unnecessary overheads.

With the API and server-side logic in place, the focus now shifts to the Frontend, where user-facing interactions, state management, and playback handling are implemented.

A Appendix

A.1 Survey Questionnaire

Music Consumption Study

This short form is designed to analyze the people's behaviour related to music - ways of discovering new audio pieces, the role of music in social interactions, preferred ways of consuming audio information etc.

1. How old are you?

   Mark only one oval.

   - ◯ 0-18 years old
   - ◯ 18-30 years old
   - ◯ 30-60 years old
   - ◯ 60+ years old

2. How do you mostly consume audio media?

   Tick all that apply.

   - ☐ Streaming platforms - Spotify, Apple Music, Yandex Music and others.
   - ☐ Vinyl, CD discs and other physical media.
   - ☐ Downloaded files.
   - ☐ Other: _____

3. How often do you listen to music?

   Tick all that apply.

   - ☐ Everyday.
   - ☐ Often, but not everyday - e.g. 3-5 days a week.
   - ☐ Once or twice a week at most.
   - ☐ Only on special occasions.
   - ☐ Other: _____

4. Roughly - how much songs do you listen to in a month?

Tick all that apply.

- [ ] 1000+
- [ ] 500-1000
- [ ] 100-500
- [ ] 50-100
- [ ] 10-50
- [ ] Less than 10
- [ ] Other: _____

5. Would you say that you are consistent with the music you listen to? I.e. do you mostly listen to the same artists/songs/genres?

Mark only one oval.

- ◯ Yes
- ◯ Mostly
- ◯ Rather no than yes
- ◯ No

6. How often do you listen to something new?

Mark only one oval.

- ◯ Very rarely, once a couple of months.
- ◯ Rarely, maybe a handful times a month.
- ◯ Pretty often, every week or so.
- ◯ Often, a couple of times a week.
- ◯ Every or almost every day.

7. How do you find out about new music?

Tick all that apply.

- [ ] At random - in cafes, shops and other venues.
- [ ] From people I know.
- [ ] Non-specialized online platforms - Instagram, TikTok, Youtube(videos), etc.
- [ ] Specialized online platforms - Rate Your Music, Pitchfork, Wire, etc.
- [ ] Games, movies, other forms of digital content.
- [ ] Buying new physical releases(vinyl, cds ...)
- [ ] Recommendations on streaming platforms
- [ ] Other: _____

8. Would you say that you know a lot of people with similar music taste?

Mark only one oval.

- ◯ Yes
- ◯ No

9. Would you like to connect with others who share your musical taste, or influence those around you to explore and appreciate the music you enjoy?

Mark only one oval.

- ◯ Yes
- ◯ No, I already have enough people around me with whom I can share the music I like
- ◯ No, I do not feel the need in that
- ◯ Other: _____

10. Are there any situations, when you listen to music with someone else?

Tick all that apply.

- [ ] At parties; when I'm visiting someone; in a car.
- [ ] Online via Spotify Jam, Discord bots and other similar instruments.
- [ ] Concerts, clubs, street performances.
- [ ] At work.
- [ ] Other: _____

11. How often do you listen to music with someone else?

Mark only one oval.

- ◯ Very rarely, maybe a couple times a year.
- ◯ Rarely, every one or two months.
- ◯ Often, a couple of times a month.
- ◯ Very often, every week or so.
- ◯ Almost every day.
- ◯ Other: _____

12. How often do you/your friends share/discuss music?

Mark only one oval.

- ◯ Very rarely, maybe a couple times a year.
- ◯ Rarely, every one or two months.
- ◯ Often, a couple of times a month.
- ◯ Very often, every week or so.
- ◯ Almost every day.
- ◯ Other: _____

13. How does that happen?

Tick all that apply.

- [ ] Online, sending links from streaming platforms via messengers.
- [ ] Offline, in a car, at someone's place etc.
- [ ] Via sharing the physical media(e.g. giving a vinyl to a friend for a listen)
- [ ] Other: _____

14. Question for users of streaming platforms.

What platform do you use?

Tick all that apply.

- [ ] Spotify
- [ ] Apple Music
- [ ] Yandex Music
- [ ] VK Music
- [ ] SoundCloud
- [ ] Bandcamp
- [ ] Youtube Music
- [ ] Other: _____

15. Question for users of streaming platforms.

Would you benefit, if current streaming platforms added extra social features(i.e. would you use them)? For example, chat discussions for playlists and songs, feeds with music added by the people you follow, forums, direct messages, etc.

Mark only one oval.

- ( ) I would use these features often.
- ( ) I would use these features rarely.
- ( ) I wouldn't use these features.

16. Question for users of streaming platforms.

Are there any other features you would like to see?

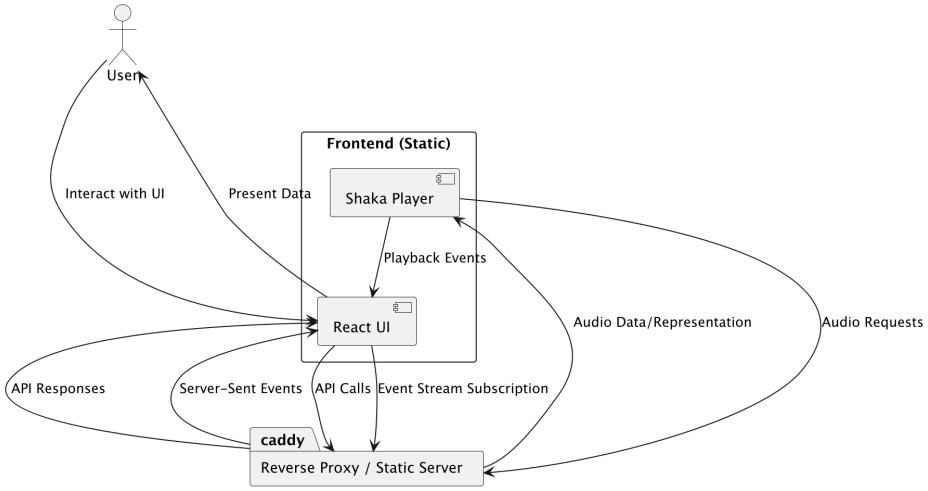

Figure 6.4: Frontend

Bibliography

[1] *Music Streaming Market Decade Insights*. URL: https://www.businessofapps.com/data/music-streaming-market/ (visited on 11/17/2025).

[2] Peter J. Rentfrow. *The Role of Music in Everyday Life: Current Directions in the Social Psychology of Music*. 2012. URL: https://compass.onlinelibrary.wiley.com/doi/abs/10.1111/j.1751-9004.2012.00434.x (visited on 03/15/2025).

[3] *Sharing Music: Social and Communal Aspects of Concert-Going*. URL: https://ojs.meccsa.org.uk/index.php/netknow/article/view/428/250 (visited on 11/17/2025).

[4] *Google Forms*. URL: https://docs.google.com/forms/u/0/ (visited on 04/05/2025).

[5] *Spotify Popularity*. URL: https://simplebeen.com/music-streaming-statistics/?utm_source=chatgpt.com (visited on 04/15/2025).

[6] *Audio Usage Statistics 2024*. URL: https://www.ifpi.org/wp-content/uploads/2023/12/IFPI-Engaging-With-Music-2023_full-report.pdf (visited on 03/15/2025).

[7] *Spotify Jam*. URL: https://support.spotify.com/cz/article/jam/ (visited on 03/16/2025).

[8] *Spotify Friend Activity*. URL: https://support.spotify.com/cz/article/friend-activity/ (visited on 03/16/2025).

[9] *Spotify Party Announcement*. URL: https://newsroom.spotify.com/2024-09-12/fans-artists-connections-features/ (visited on 03/16/2025).

[10] *Spotify Party*. URL: https://support.spotify.com/us/article/listening-party/ (visited on 03/16/2025).

[11] *Spotify Chat*. URL: https://newsroom.spotify.com/2025-08-26/introducing-messages-a-new-way-to-share-what-you-love-on-spotify-with-friends-and-family/ (visited on 11/17/2025).

[12] *VK Music*. URL: https://music.vk.com/ (visited on 11/17/2025).

[13] *SoundCloud Comments*. URL: https://help.soundcloud.com/hc/en-us/articles/115003566008-Commenting-basics (visited on 03/16/2025).

[14] *SoundCloud Reactions*. URL: https://help.soundcloud.com/hc/en-us/articles/31039497560731-Reactions (visited on 03/16/2025).

[15] *Soundcloud Reposts*. URL: https://help.soundcloud.com/hc/en-us/articles/115003567488-Reposting-tracks-or-playlists (visited on 03/16/2025).

[16] *Rate Your Music*. URL: https://rateyourmusic.com/discover (visited on 03/25/2025).

[17] *Instagram Audio*. URL: https://help.instagram.com/664508917489819?helpref=faq_content (visited on 03/25/2025).

[18] Dai Clegg and Richard Barker. *Case Method Fast-Track: A Rad Approach*. URL: https://dl.acm.org/doi/10.5555/561543 (visited on 05/05/2025).

[19] *Multipurpose Internet Mail Extensions Part Two: Media Types. 'octet-stream'*. URL: https://www.rfc-editor.org/rfc/rfc2046#section-4.5.1 (visited on 11/30/2025).

[20] *Nginx pseudo-streaming implementation*. URL: https://nginx.org/en/docs/http/ngx_http_mp4_module.html (visited on 11/30/2025).

[21] *HTTP Range Requests*. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Range_requests (visited on 12/07/2025).

[22] Issing Jochen Färber Nikolaus Döhla Stefan. *Adaptive progressive download based on the MPEG-4 file format*. URL: https://link.springer.com/article/10.1631/jzus.2006.AS0106 (visited on 11/30/2025).

[23] *The MPEG-DASH Standard for Multimedia Streaming Over the Internet*. URL: https://ieeexplore.ieee.org/abstract/document/6077864 (visited on 11/28/2025).

[24] *AWS CDN Caching*. URL: https://aws.amazon.com/caching/ (visited on 12/14/2025).

[25] *Basic Authentication*. URL: https://datatracker.ietf.org/doc/html/rfc7617 (visited on 03/19/2025).

[26] *Mozilla Caching Behaviour Bug Tracker*. URL: https://bugzilla.mozilla.org/show_bug.cgi?id=300002 (visited on 11/01/2025).

[27] *Mozilla Caching Behaviour Documentation*. URL: https://support.mozilla.org/en-US/kb/delete-browsing-search-download-history-firefox (visited on 11/01/2025).

[28] *Session Authentication*. URL: https://docs.djangoproject.com/en/5.2/topics/http/sessions/ (visited on 03/19/2025).

[29] *Redis*. URL: https://redis.io/ (visited on 12/07/2025).

[30] *OAuth2*. URL: https://datatracker.ietf.org/doc/html/rfc6749 (visited on 03/21/2025).

[31] *Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP*. URL: https://datatracker.ietf.org/doc/html/rfc6202 (visited on 11/30/2025).

[32] *Comparison of Server to Client Communication Methods*. URL: https://www.diva-portal.org/smash/get/diva2:1133465/FULLTEXT01.pdf?ref=hackernoon.com (visited on 03/29/2025).

[33] *Server Sent Events*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events (visited on 05/02/2025).

[34] *Django Channels*. URL: https://channels.readthedocs.io/en/latest/ (visited on 03/19/2025).

[35] *WebSockets*. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API (visited on 05/02/2025).
[36] *Http Caching*. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching (visited on 05/02/2025).
[37] *Push API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Push_API (visited on 05/02/2025).
[38] *Generic Event Delivery Using HTTP Push*. URL: https://datatracker.ietf.org/doc/html/rfc8030 (visited on 11/30/2025).
[39] *Service Worker API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API (visited on 11/30/2025).
[40] *Java, Python and Javascript, a comparison*. URL: https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1355073&dswid=-4197 (visited on 12/07/2025).
[41] *JavaScript*. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript (visited on 12/07/2025).
[42] *FastAPI*. URL: https://fastapi.tiangolo.com/ (visited on 03/18/2025).
[43] *Pydantic*. URL: https://docs.pydantic.dev/latest/ (visited on 05/01/2025).
[44] *FastAPI Features*. URL: https://fastapi.tiangolo.com/features (visited on 03/20/2025).
[45] *FastAPI Websockets*. URL: https://fastapi.tiangolo.com/advanced/websockets/ (visited on 12/01/2025).
[46] *StackOverflow 2024 Developers Survey*. URL: https://survey.stackoverflow.co/2024/technology (visited on 12/01/2025).
[47] *Django*. URL: https://www.djangoproject.com/ (visited on 03/18/2025).
[48] *DRF*. URL: https://www.django-rest-framework.org/ (visited on 03/19/2025).
[49] *Django Overview*. URL: https://www.djangoproject.com/start/overview/ (visited on 12/07/2025).
[50] *Django Async Support*. URL: https://docs.djangoproject.com/en/5.2/topics/async/ (visited on 01/30/2025).
[51] *Celery Library*. URL: https://docs.celeryq.dev (visited on 12/07/2025).

[52] *Flask*. URL: https://flask.palletsprojects.com/en/stable/ (visited on 03/19/2025).
[53] *SocketIO*. URL: https://socket.io/ (visited on 12/01/2025).
[54] *Web Service Efficiency at Instagram with Python*. URL: https://instagram-engineering.com/web-service-efficiency-at-instagram-with-python-4976d078e366 (visited on 12/07/2025).
[55] *Oracle Price List*. URL: https://www.oracle.com/a/ocom/docs/corporate/pricing/technology-price-list-070617.pdf (visited on 11/30/2025).
[56] *PostgreSQL Full Text Search*. URL: https://www.postgresql.org/docs/current/textsearch.html (visited on 12/07/2025).
[57] *PostgreSQL*. URL: https://www.postgresql.org/about/ (visited on 03/26/2025).
[58] *Django PostgreSQL Support*. URL: https://docs.djangoproject.com/en/5.2/ref/contrib/postgres/ (visited on 03/26/2025).
[59] *Postgres UUID Type*. URL: https://www.postgresql.org/docs/current/datatype-uuid.html (visited on 12/01/2025).
[60] *Django Templates*. URL: https://docs.djangoproject.com/en/5.2/ref/templates/language/ (visited on 01/30/2025).
[61] *Single Page Application*. URL: https://developer.mozilla.org/en-US/docs/Glossary/SPA (visited on 12/07/2025).
[62] *Clay Library*. URL: https://github.com/nicbarker/clay (visited on 12/01/2025).
[63] *Yew Framework*. URL: https://github.com/yewstack/yew (visited on 12/01/2025).
[64] *Framework Usage*. URL: https://2024.stateofjs.com/en-US/libraries/ (visited on 05/01/2025).
[65] *React*. URL: https://react.dev/ (visited on 03/30/2025).
[66] *How declarative UI compares to imperative*. URL: https://react.dev/learn/reacting-to-input-with-state#how-declarative-ui-compares-to-imperative (visited on 12/07/2025).
[67] *React Developer Tools*. URL: https://react.dev/learn/react-developer-tools (visited on 05/01/2025).

[68] *StackOverflow React*. URL: https://stackoverflow.com/questions/tagged/reactjs?tab=Newest (visited on 12/07/2025).
[69] *Addressing the Limitations of React JS*. URL: https://www.academia.edu/44039524/IRJET_Addressing_the_Limitations_of_React_JS (visited on 12/07/2025).
[70] *VueJS*. URL: https://vuejs.org/about/faq.html (visited on 03/30/2025).
[71] *Angular*. URL: https://angular.dev/ (visited on 12/07/2025).
[72] *MVC*. URL: https://developer.mozilla.org/en-US/docs/Glossary/MVC (visited on 03/18/2025).
[73] *MPEG-DASH*. URL: https://www.mpeg.org/standards/MPEG-DASH/ (visited on 05/05/2025).
[74] *HLS*. URL: https://developer.apple.com/streaming/ (visited on 05/05/2025).
[75] *Comparative Analysis of MPEG-DASH and HLS Protocols: Performance, Adaptation, and Future Directions in Adaptive Streaming*. URL: https://link.springer.com/article/10.1007/s40031-025-01244-x#Sec4 (visited on 05/05/2025).
[76] *MSE*. URL: https://www.w3.org/TR/media-source-2/ (visited on 05/05/2025).
[77] *MSE Availability*. URL: https://caniuse.com/mediasource (visited on 05/05/2025).
[78] *Operating System Market Share Worldwide*. URL: https://gs.statcounter.com/os-market-share (visited on 12/13/2025).
[79] *Packaging explanation*. URL: https://shaka-project.github.io/shaka-packager/html/documentation.html (visited on 12/13/2025).
[80] *CMAF*. URL: https://developer.apple.com/documentation/http-live-streaming/about-the-common-media-application-format-with-http-live-streaming-hls (visited on 12/13/2025).
[81] *HTTP Live Streaming (HLS) authoring specification for Apple devices. Audio.* URL: https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices#Audio (visited on 12/13/2025).
[82] *FLAC*. URL: https://xiph.org/flac/ (visited on 12/14/2025).

[83] *ISO/IEC 11172-3:1993 (MP3(*. URL: https://www.iso.org/standard/22412.html (visited on 12/14/2025).

[84] *ISO/IEC 13818-7:2006 (AAC)*. URL: https://www.iso.org/standard/43345.html (visited on 12/14/2025).

[85] *Opus Codec*. URL: https://opus-codec.org/ (visited on 05/06/2025).

[86] *EBU Evaluations of Mutlichannel Audio Codecs*. URL: https://tech.ebu.ch/docs/tech/tech3324.pdf (visited on 12/14/2025).

[87] *Perceived Audio Quality for Streaming Stereo Music*. URL: https://dl.acm.org/doi/pdf/10.1145/2647868.2655025 (visited on 12/13/2025).

[88] *BACHELOR THESIS Perceived Audio Quality in Ogg Vorbis and AAC in Compressed Popular Music in Bit Rates between 96 kbit/s, 128 kbit/s and 192 kbit/s*. URL: https://www.diva-portal.org/smash/get/diva2:1030609/FULLTEXT02.pdf (visited on 12/14/2025).

[89] *HTTP Live Streaming (HLS) authoring specification for Apple devices. Bitrate recommendations, section 2.25.* URL: https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices#Bitrate-recommendations-for-Ambisonics-are-as-follows (visited on 12/13/2025).

[90] *CBR*. URL: https://learn.microsoft.com/en-us/windows/win32/wmformat/constant-bit-rate--cbr--encoding (visited on 05/06/2025).

[91] *VBR*. URL: https://wiki.hydrogenaudio.org/index.php?title=Variable_Bitrate (visited on 05/06/2025).

[92] *CBR vs VBR*. URL: https://ieeexplore.ieee.org/abstract/document/6774590 (visited on 05/06/2025).

[93] *Spotify Audio Quality*. URL: https://support.spotify.com/cz/article/audio-quality/ (visited on 12/14/2025).

[94] *Web audio codec guide*. URL: hhttps://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats/Audio_codecs#aac_advanced_audio_coding (visited on 12/14/2025).

[95] *FFMPEG*. URL: https://ffmpeg.org/ (visited on 05/05/2025).

[96] *ffmpeg libfdkaac*. URL: https://trac.ffmpeg.org/wiki/Encode/AAC (visited on 05/06/2025).

[97] *ffmpeg flac codec*. URL: https://ffmpeg.org/ffmpeg-codecs.html#toc-flac-2 (visited on 12/14/2025).

[98] *Movie atom ('moov')*. URL: https://developer.apple.com/documentation/quicktime-file-format/movie_atom (visited on 12/14/2025).

[99] *Shaka Player*. URL: https://github.com/shaka-project/shaka-player (visited on 05/08/2025).

[100] *Shaka Packager*. URL: https://shaka-project.github.io/shaka-packager/html/ (visited on 12/13/2025).

[101] *REST Definition*. URL: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (visited on 05/07/2025).

[102] *python-magic Package*. URL: https://pypi.org/project/python-magic/ (visited on 05/07/2025).

[103] *Django Content Type Framework*. URL: https://docs.djangoproject.com/en/5.2/ref/contrib/contenttypes/ (visited on 05/07/2025).